

Artificial Intelligence

Instructor: Kietikul Jearanaitanakij

Department of Computer Engineering

King Mongkut's Institute of Technology Ladkrabang

Learning from examples (Decision Tree Learning)

- Forms of Learning
- Supervised Learning
- Learning Decision Trees
- Evaluating the Hypothesis

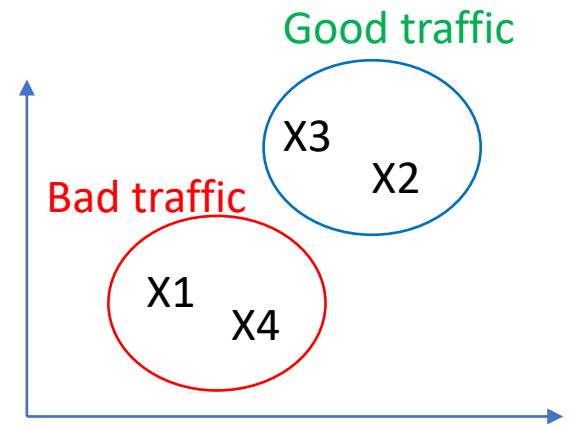
Forms of Learning

1. Supervised learning

- The agent observes some example input-output pairs and learns a function that maps from input to output.

Example

A taxi agent learns to predict “Good traffic” and “Bad traffic” from the “input-output” examples.



- X1: Rain, BigEvent, WorkDay, MonthEnd, “Bad traffic”
- X2: NonRain, NonBigEvent, NonWorkDay, NonMonthEnd, “Good traffic”
- X3: Rain, NonBigEvent, WorkDay, NonMonthEnd, “Good traffic”
- X4: NonRain, BigEvent, NonWorkDay, MonthEnd, “Bad traffic”

Features / Attributes

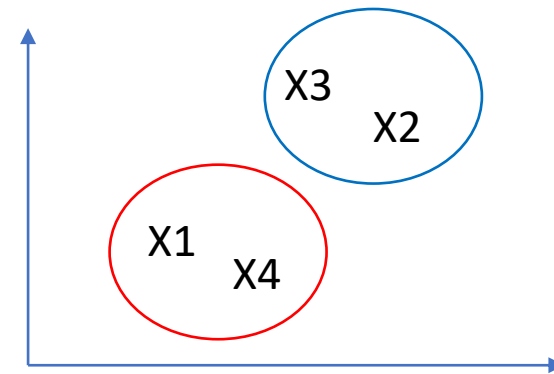
Target Class / Label / Ground truth

2. Unsupervised learning

- The agent learns patterns in the input without any label.
- The most common unsupervised learning task is **clustering**.

Example (Online shopping)

- **X1: Shirt, Suit, Necktie**
- **X2: Running Shoe, Shorts, Sock**
- **X3: Running Shoe, Sport watch, Sock**
- **X4: Shirt, Trousers, Necktie**



Customers X1 and X4 share the same cluster whereas X2 and X3 are in another cluster.

3. Reinforcement learning

- There is no training data available.
- The agent must gradually learn from the real situation (a series of reinforcements).
- At the end of each run, the agent is given either **reward** or **punishment**.

Example:

The **lack of a tip** at the end of the journey gives the taxi agent an indication that it did something wrong.

It is up to the agent to decide which of the actions before the reinforcement was most responsible for it.

Supervised Learning

Problem statement

Given a training set of N example input-output pairs

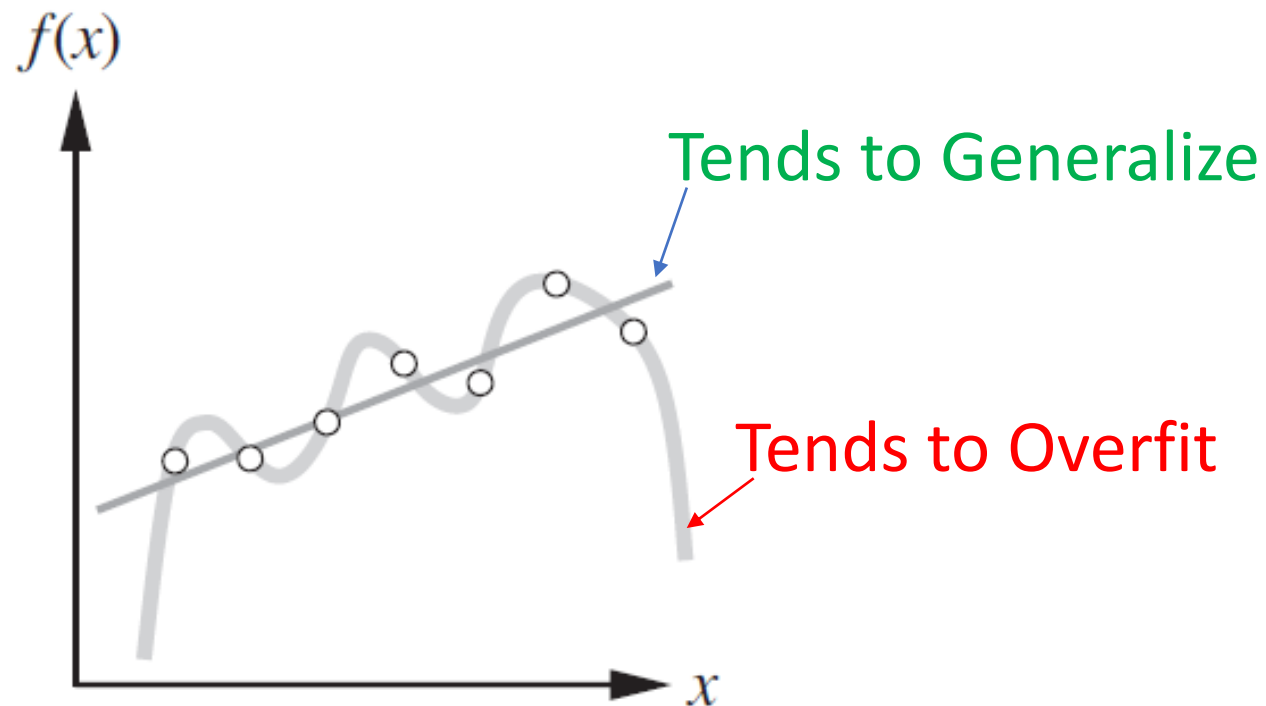
$$(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N),$$

Find the **hypothesis function h** that approximates the true function of the training set.

- If **y** is one of a **finite set of values, e.g., $\{C_1, C_2, C_3\}$** , the learning problem is called **classification**.
- If **y** is a **real number** (such as tomorrow's temperature), the learning problem is called **regression**.
- To evaluate a hypothesis function, we test it on an unseen dataset (**test set**)
- We say a hypothesis **generalizes** well if it correctly predicts the unseen data.

Overfitting and Generalization

- Overfitting occurs when a learning model learns noises instead of just the actual data.
- An **overfitted** model generally has a poor prediction.
- A good learning model should be **generalized** to both unseen and training data.



Learning Decision Trees

(ID3 algorithm, Iterative Dichotomiser 3)

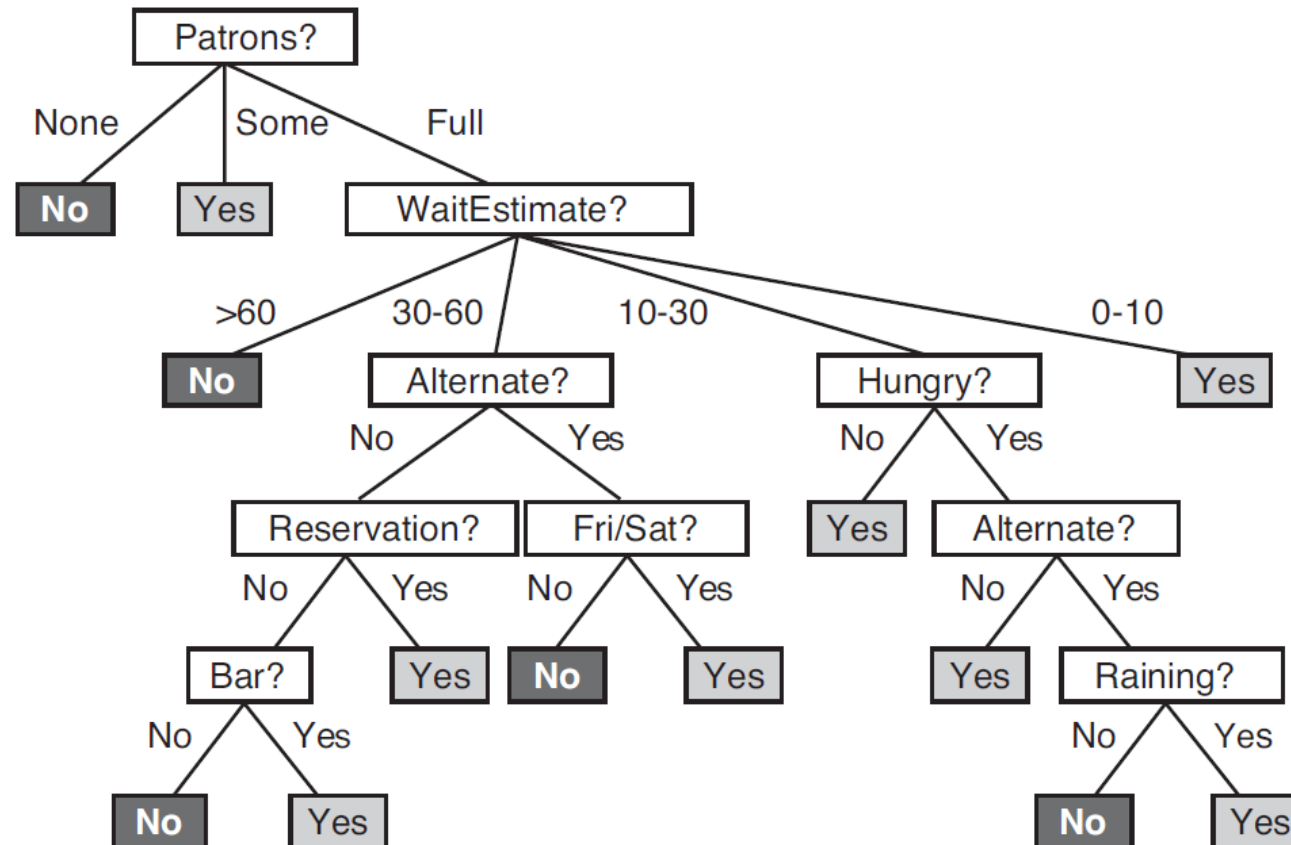
For example, we will build a decision tree to decide whether to wait for a table at a restaurant.

- The objective is to learn a goal predicate **WillWait**.
 - The following attributes may influence the customer's decision.
1. **Alternate**: whether there is a suitable alternative restaurant nearby.
 2. **Bar** : whether the restaurant has a comfortable bar area to wait in.
 3. **Fri/Sat**: true on Fridays and Saturdays.
 4. **Hungry**: whether we are hungry.
 5. **Patrons**: how many people are in the restaurant (None, Some, and Full).
 6. **Price**: the restaurant's price range.
 7. **Raining**: whether it is raining outside.
 8. **Reservation**: whether there is a reservation.
 9. **Type**: the kind of restaurant (French, Italian, Thai, or burger).
 10. **WaitEstimate**: the wait estimated by the host (0–10 minutes, 10–30, 30–60, or >60).

Examples for the restaurant domain.

Example	Input Attributes										Goal
	<i>Alt</i>	<i>Bar</i>	<i>Fri</i>	<i>Hun</i>	<i>Pat</i>	<i>Price</i>	<i>Rain</i>	<i>Res</i>	<i>Type</i>	<i>Est</i>	<i>WillWait</i>
x₁	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>French</i>	<i>0–10</i>	$y_1 = \text{Yes}$
x₂	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Thai</i>	<i>30–60</i>	$y_2 = \text{No}$
x₃	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Some</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Burger</i>	<i>0–10</i>	$y_3 = \text{Yes}$
x₄	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Thai</i>	<i>10–30</i>	$y_4 = \text{Yes}$
x₅	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>Full</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>French</i>	<i>>60</i>	$y_5 = \text{No}$
x₆	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$</i>	<i>Yes</i>	<i>Yes</i>	<i>Italian</i>	<i>0–10</i>	$y_6 = \text{Yes}$
x₇	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>None</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Burger</i>	<i>0–10</i>	$y_7 = \text{No}$
x₈	<i>No</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$</i>	<i>Yes</i>	<i>Yes</i>	<i>Thai</i>	<i>0–10</i>	$y_8 = \text{Yes}$
x₉	<i>No</i>	<i>Yes</i>	<i>Yes</i>	<i>No</i>	<i>Full</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Burger</i>	<i>>60</i>	$y_9 = \text{No}$
x₁₀	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>Italian</i>	<i>10–30</i>	$y_{10} = \text{No}$
x₁₁	<i>No</i>	<i>No</i>	<i>No</i>	<i>No</i>	<i>None</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Thai</i>	<i>0–10</i>	$y_{11} = \text{No}$
x₁₂	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Burger</i>	<i>30–60</i>	$y_{12} = \text{Yes}$

A decision tree for deciding whether to wait for a table.

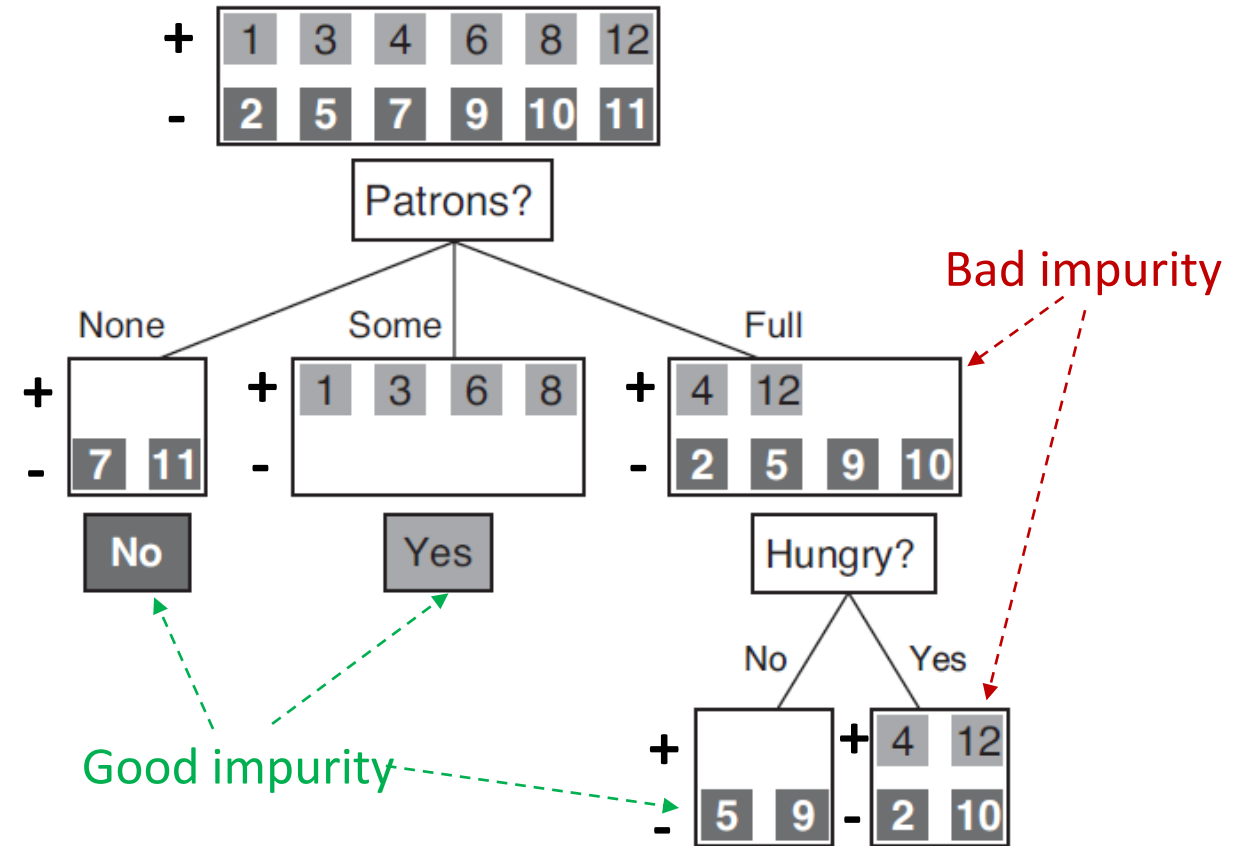
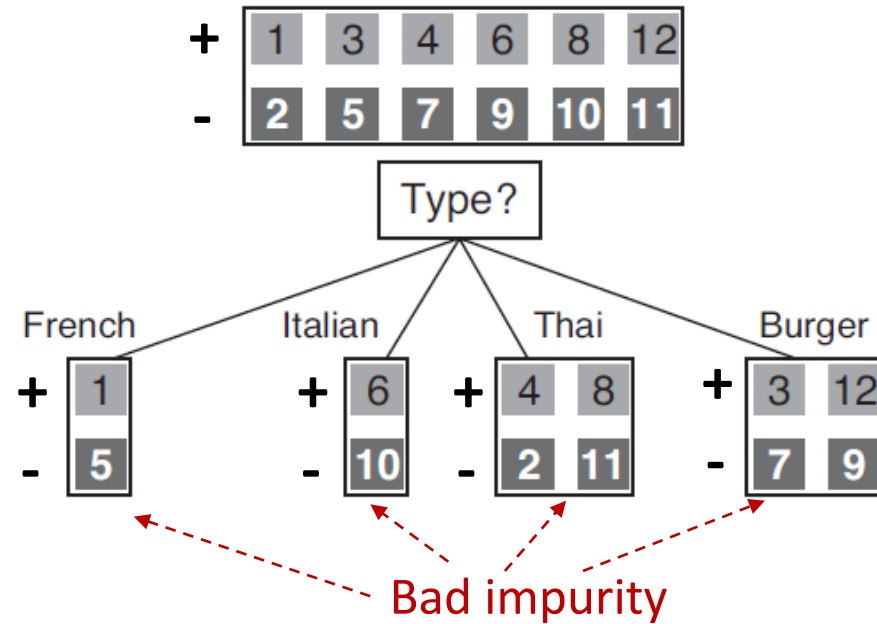


However, this decision tree is not optimal (we will see later).

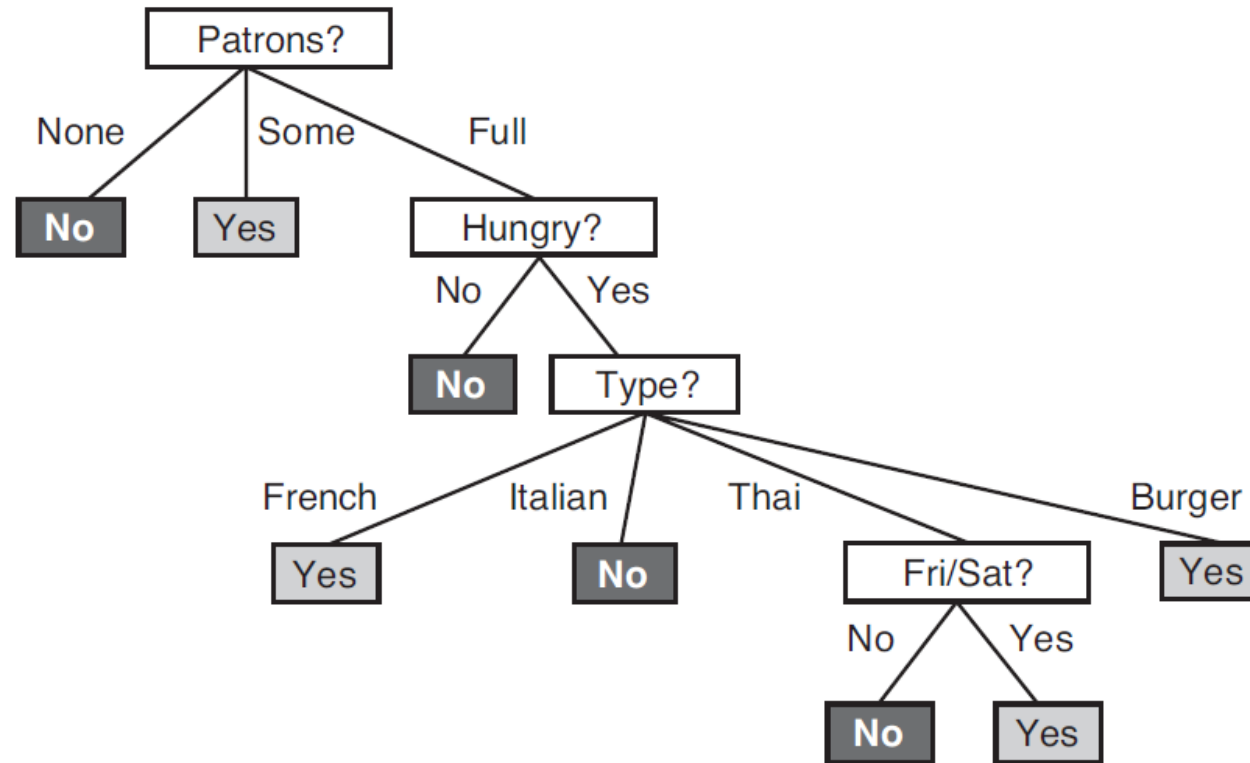
Creating the optimal decision tree

- The optimal decision tree is the **shallowest**. It will save the runtime.
- Strategy: Test **the most important attribute first**.
- The **most important attribute**:
The attribute with the **lowest impurity**, i.e., the smallest number of examples has not been decided yet.

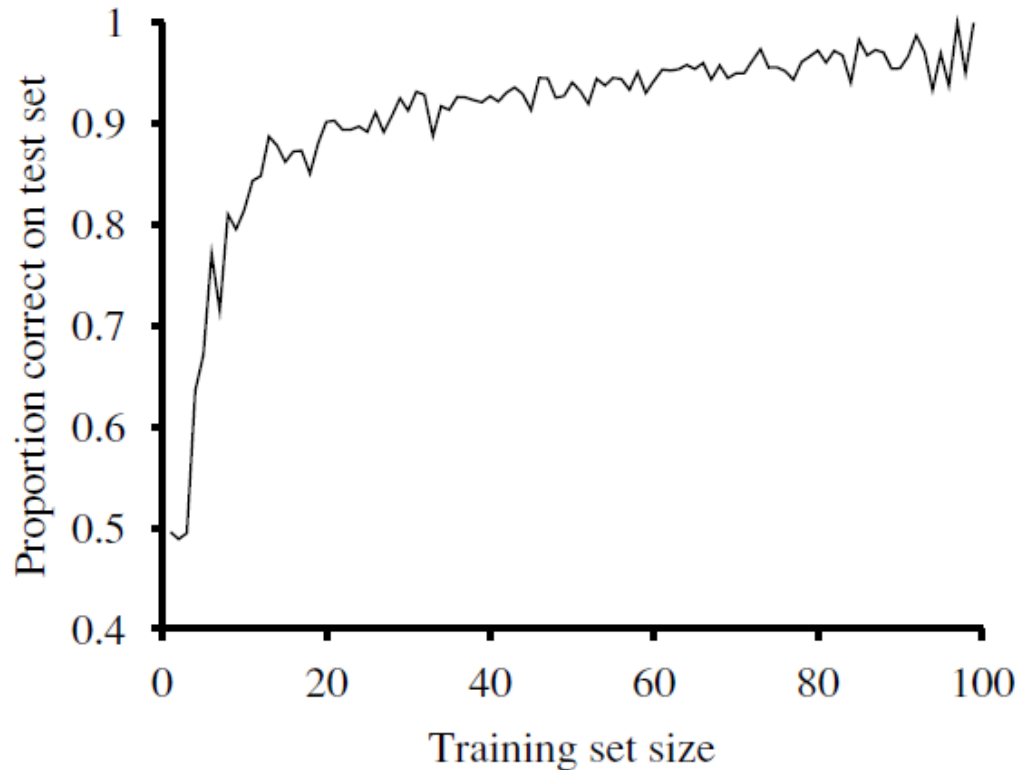
Which attribute is more important (**Type** or **Patrons**)?



The optimal decision tree induced from the 12-example training set.



- We can evaluate the accuracy of a learning algorithm with a **learning curve**. We have 100 examples at our disposal, which we split into a training set and a test set.



- We learn a hypothesis h with the training set and measure its accuracy with the test set.
- We do this starting with a training set of size 1 and increasing one at a time up to size 99. For each size we actually repeat the process of randomly splitting 20 times, and average the results of the 20 trials.
- The curve shows that as the training set size grows, the accuracy increases.

In this graph, we reach 95% accuracy, and it looks like the curve might continue to increase with more data.

Information gain and Entropy

- Counting the **impurity** of each attribute just tells us a **rough importance**.
- To have a more precise measure, we should employ an **Information gain**.
- **Information gain** is defined in terms of **Entropy** (Shannon and Weaver, 1949), to define a precise measure of an attribute's importance.
- **Entropy** is an uncertainty of a random variable.

Intuition

- If a coin is fair, its uncertainty is very high (**entropy = 1**)
- If a coin always comes up heads, it has no uncertainty (**entropy = 0**).

Entropy formula

- If a training set contains **p positive** examples and **n negative** examples,

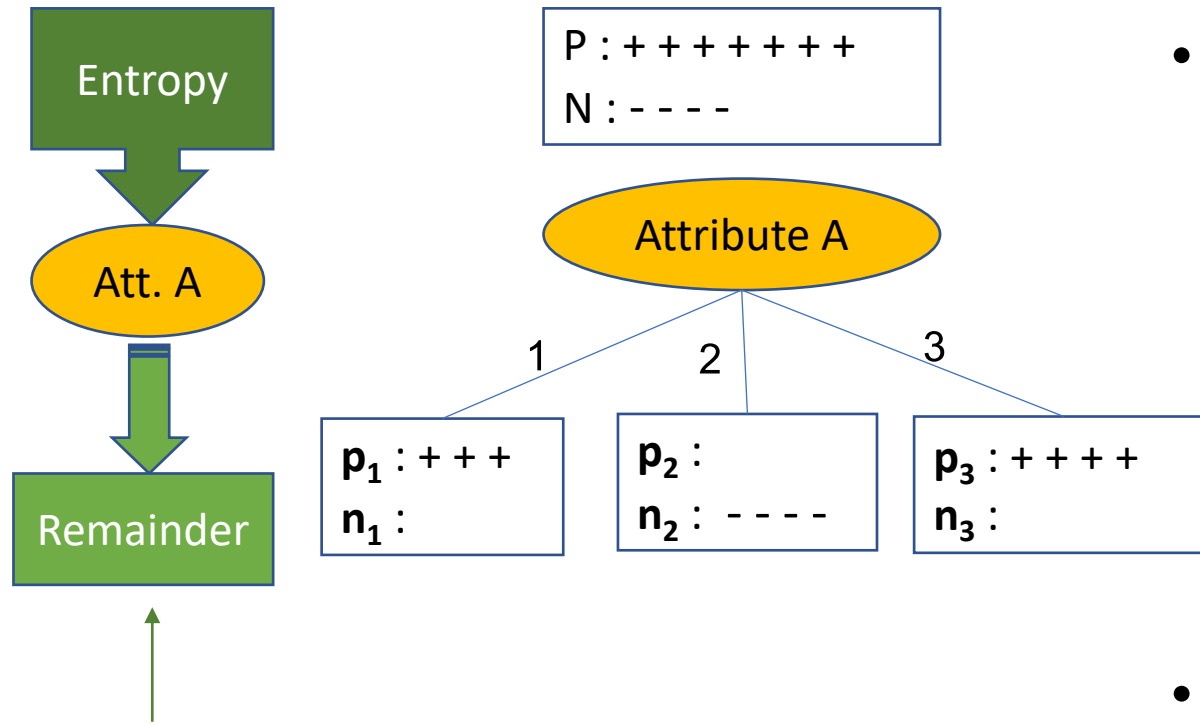
Entropy:
$$I\left(\frac{p}{p+n}, \frac{n}{p+n}\right) = -\frac{p}{p+n} \log_2 \frac{p}{p+n} - \frac{n}{p+n} \log_2 \frac{n}{p+n}$$

- We can check that the entropy of **a fair coin** flip is indeed **1**:

$$I\left(\frac{1}{1+1}, \frac{1}{1+1}\right) = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = 1.$$

- If the coin gives 99% heads, we get

$$I\left(\frac{99}{99+1}, \frac{1}{99+1}\right) = -\frac{99}{100} \log_2 \frac{99}{100} - \frac{1}{100} \log_2 \frac{1}{100} = 0.08.$$



- We can measure how much the entropy (uncertainty) remaining after testing the attribute A by

$$Remainder(A) = \sum_{i=1}^v \frac{p_i + n_i}{p + n} \cdot I\left(\frac{p_i}{p_i + n_i}, \frac{n_i}{p_i + n_i}\right)$$

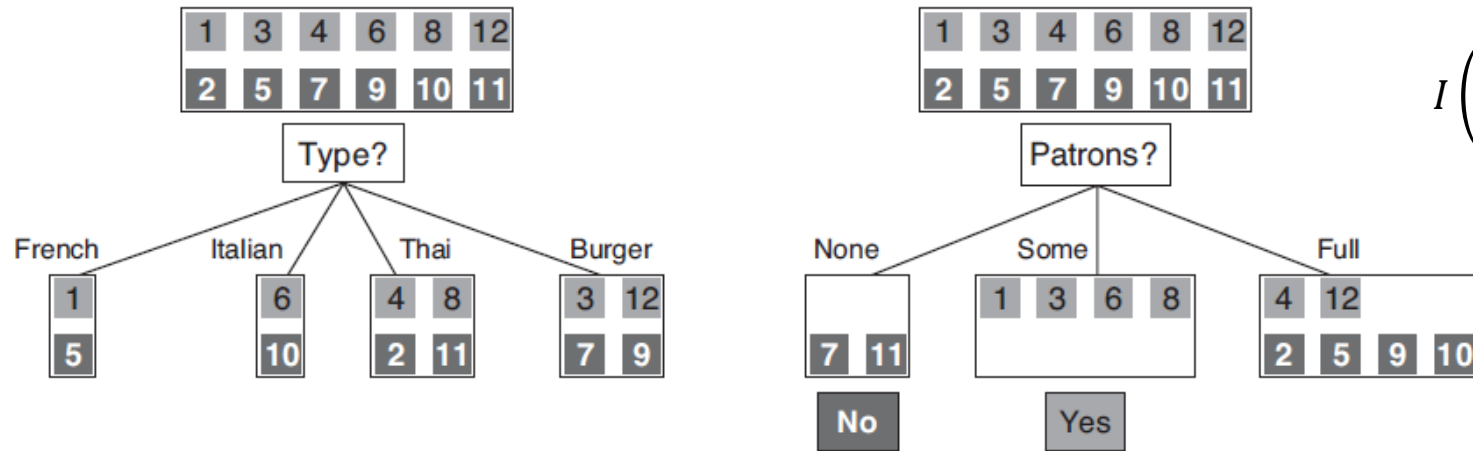
- The **information gain** from the attribute A is defined as

$$Gain(A) = I\left(\frac{p}{p + n}, \frac{n}{p + n}\right) - Remainder(A)$$

- The more value of the information gain of the attribute A, the more important it is.

The less the remainder, the more important the attribute A.

- Now, let us calculate the information of both Patrons and Type



$$I\left(\frac{p}{p+n}, \frac{n}{p+n}\right) = -\frac{p}{p+n} \log_2 \frac{p}{p+n} - \frac{n}{p+n} \log_2 \frac{n}{p+n}$$

$$Remainder(A) = \sum_{i=1}^v \frac{p_i + n_i}{p+n} \cdot I\left(\frac{p_i}{p_i+n_i}, \frac{n_i}{p_i+n_i}\right)$$

$$Gain(A) = I\left(\frac{p}{p+n}, \frac{n}{p+n}\right) - Remainder(A)$$

$$Gain(Type) = I\left(\frac{6}{6+6}, \frac{6}{6+6}\right) - \left[\frac{2}{12} \cdot I\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{2}{12} \cdot I\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{4}{12} \cdot I\left(\frac{2}{4}, \frac{2}{4}\right) + \frac{4}{12} \cdot I\left(\frac{2}{4}, \frac{2}{4}\right) \right]$$

$$= 0$$

$$Gain(Patrons) = I\left(\frac{6}{6+6}, \frac{6}{6+6}\right) - \left[\frac{2}{12} \cdot I(0,1) + \frac{4}{12} \cdot I(1,0) + \frac{6}{12} \cdot I\left(\frac{2}{6}, \frac{4}{6}\right) \right]$$

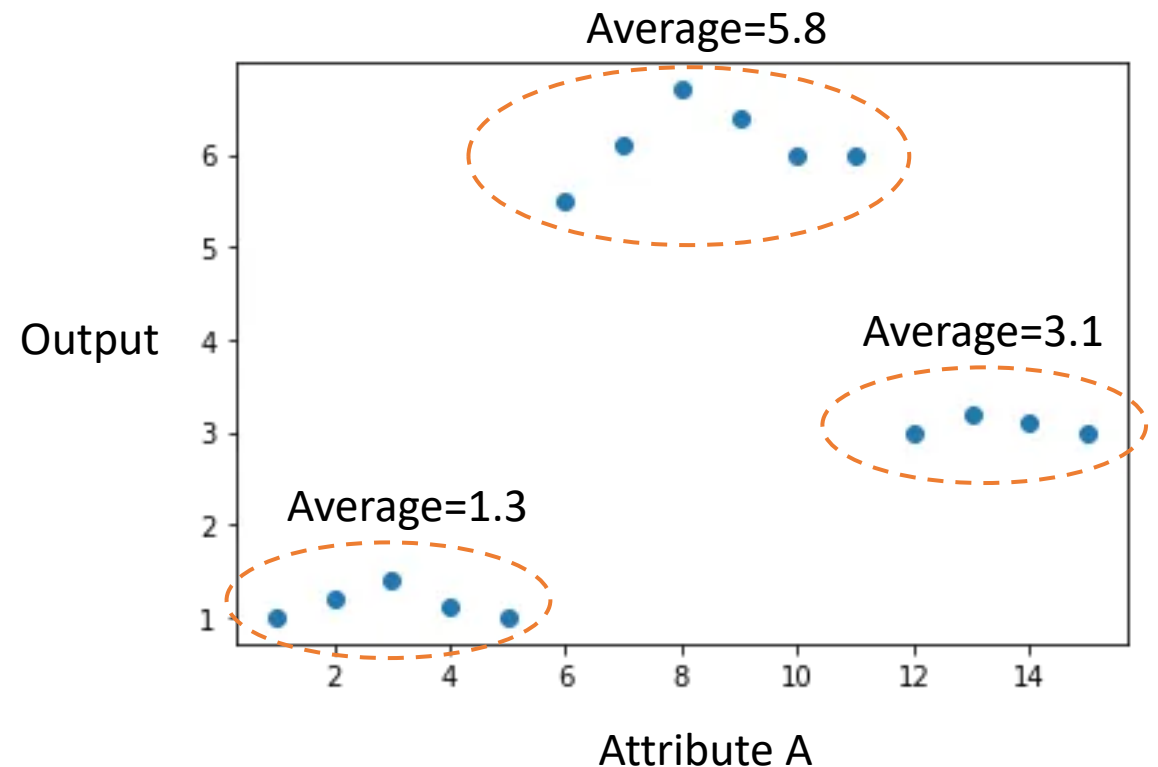
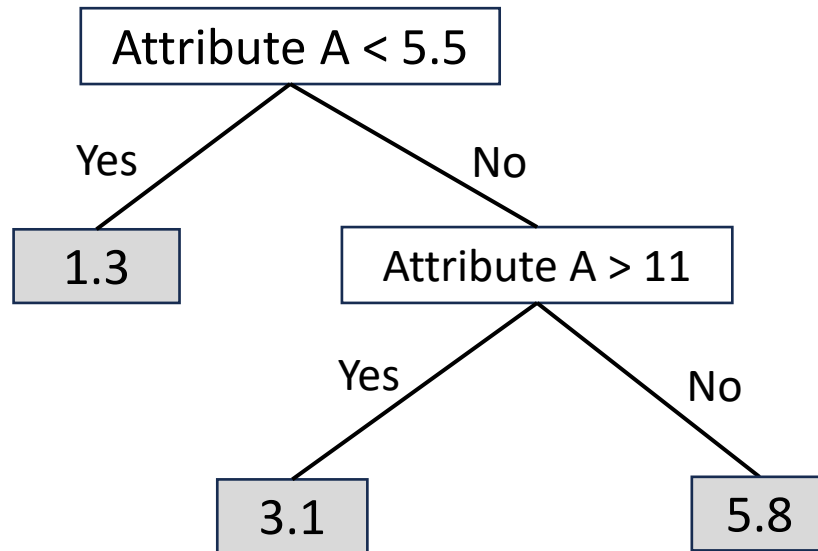
$$= 1 - [0 + 0 + 0.4594]$$

$$= 0.5406$$

Therefore, Patrons is more important than Type.

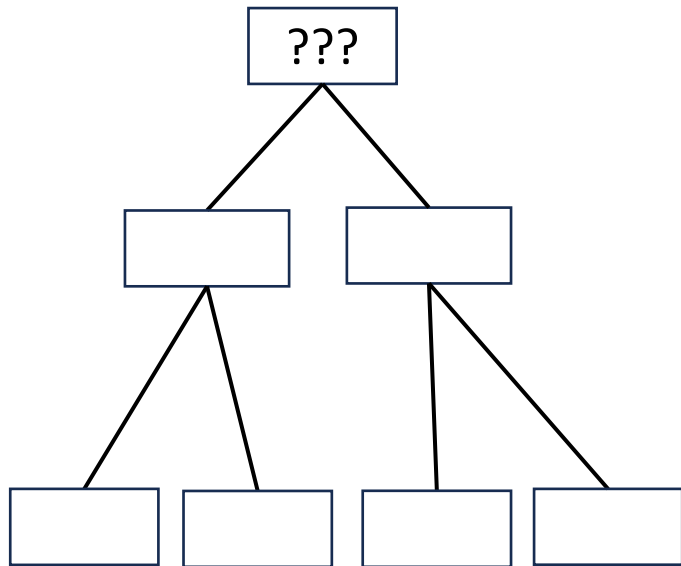
Applying ID3 to a Regression Problem (Regression Tree)

- Regression is a problem of using the input values to predict continuous valued output (instead of a discrete output like a classification problem).



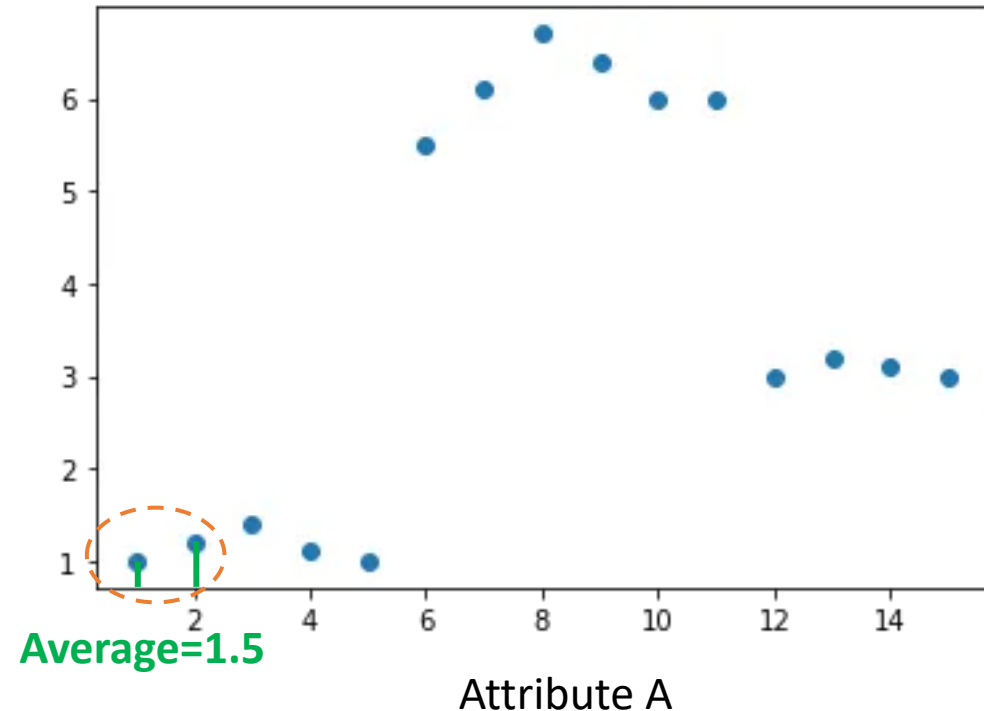
Applying ID3 to a Regression Problem (Regression Tree)

- Just like for Classification Trees, the first thing we do for a Regression Tree is decide what goes in the Root.

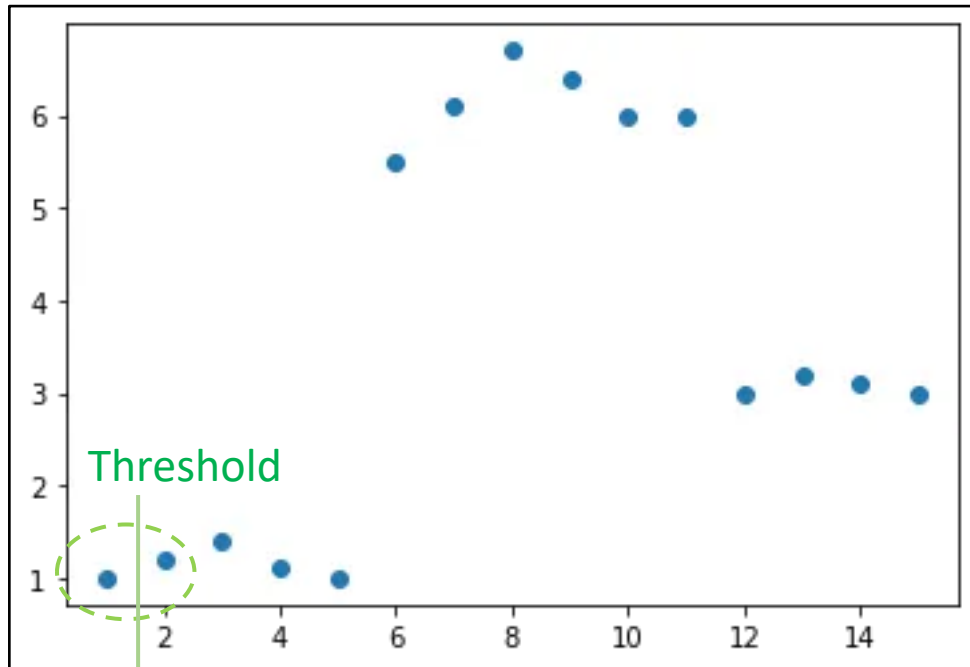


First, find the average of the first 2 points and use it to split all data points.

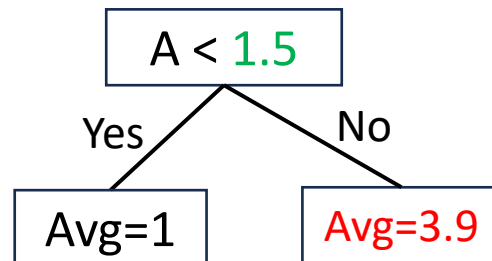
Output



Applying ID3 to a Regression Problem (Regression Tree)

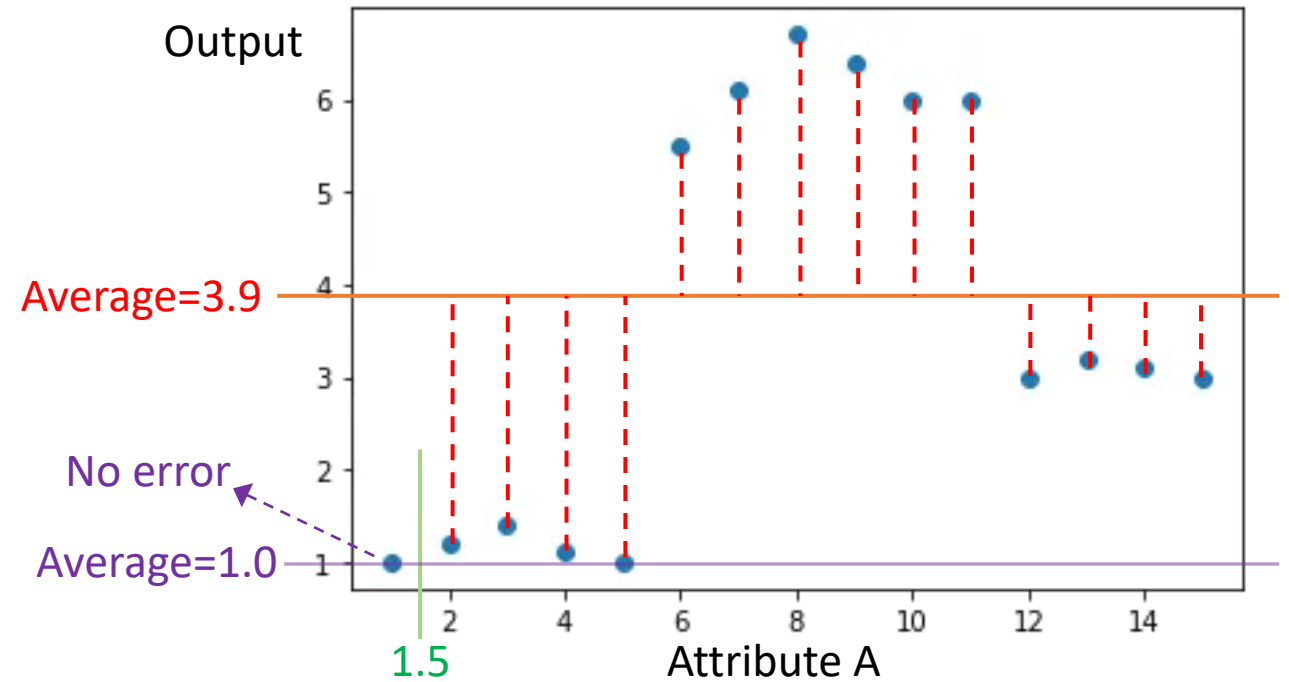


1.5



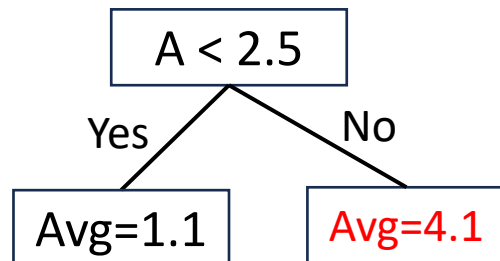
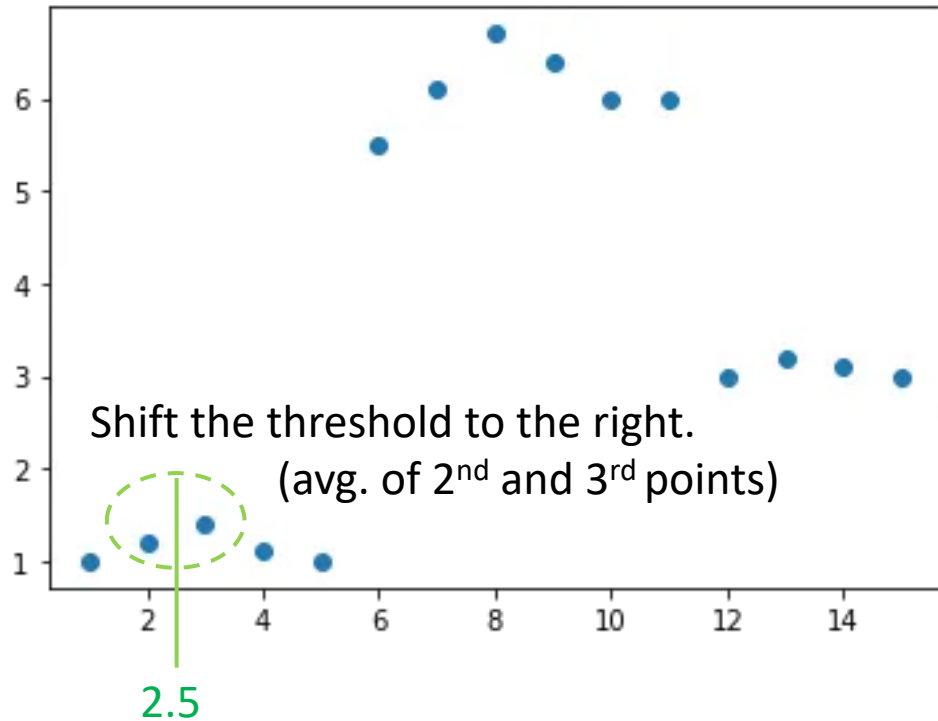
Avg. output 1 point

Avg. outputs of 14 points on the right.



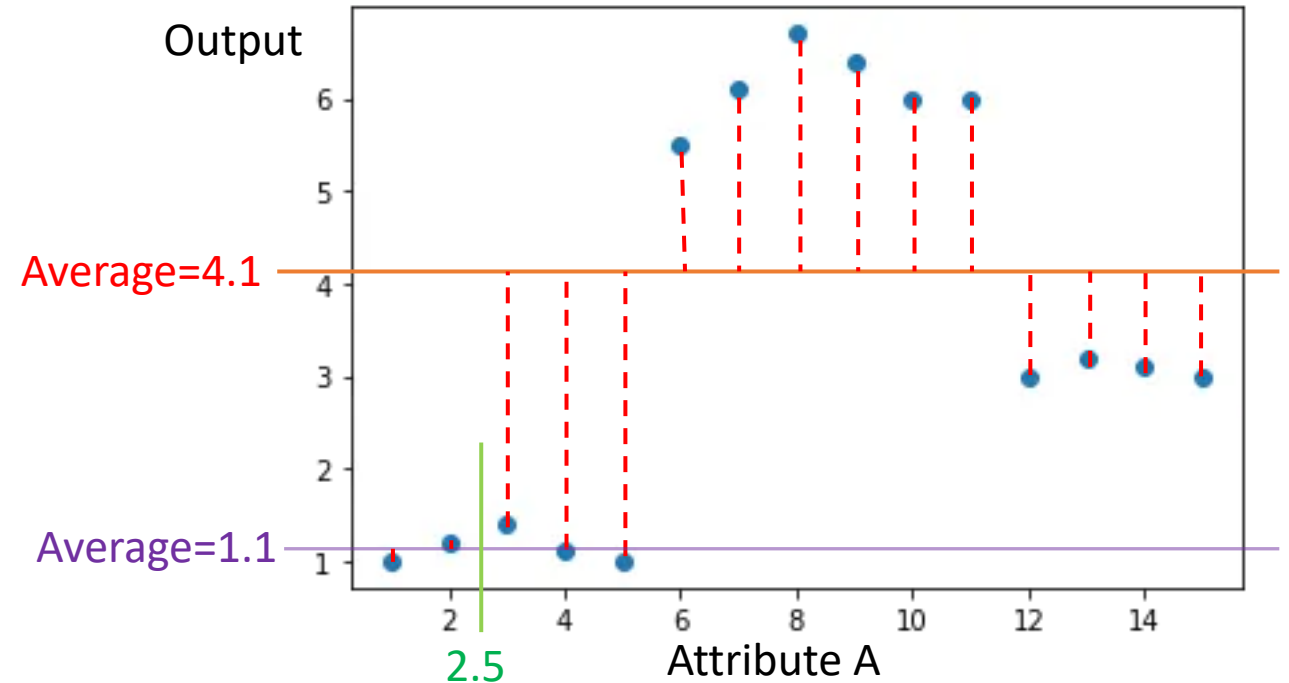
- The **dashed lines** represent **errors** between individual points on the right and the average 3.9.
- Then, calculate the sum-squared error of all data points. Let's call it **E1**.

Applying ID3 to a Regression Problem (Regression Tree)



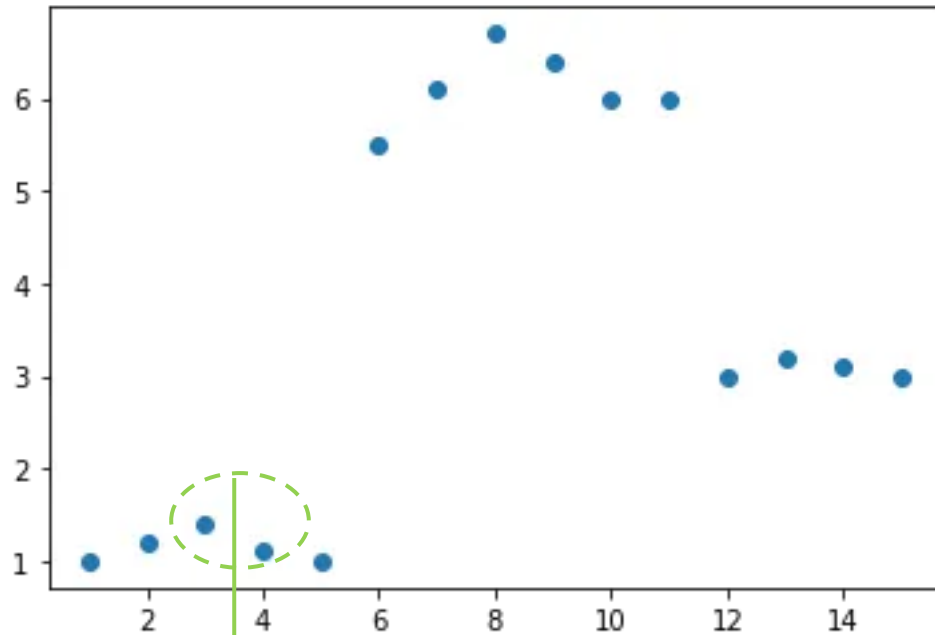
Avg. outputs of 2 points
on the left.

Avg. outputs of 13 points on the right.

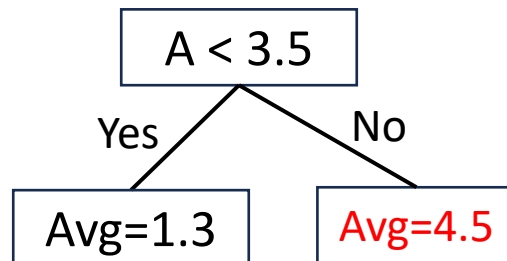


- Calculate the sum-squared error of all data points. Let's call it **E2**.

Applying ID3 to a Regression Problem (Regression Tree)

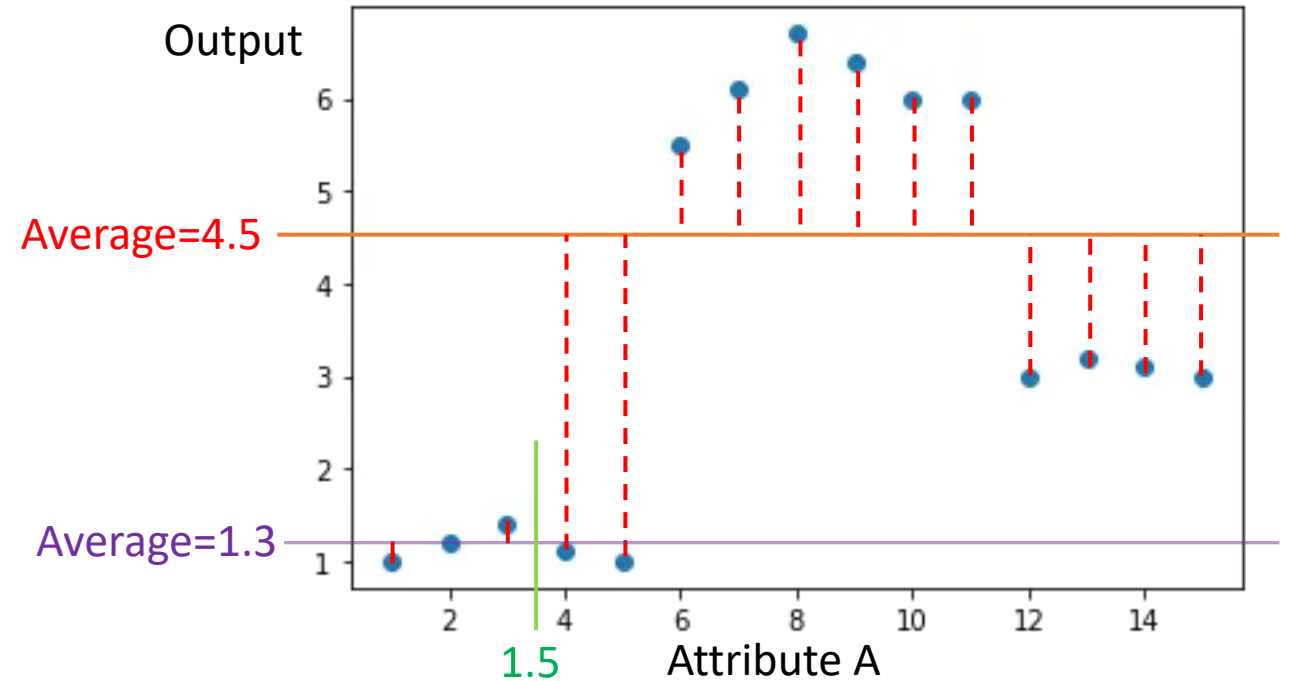


3.5



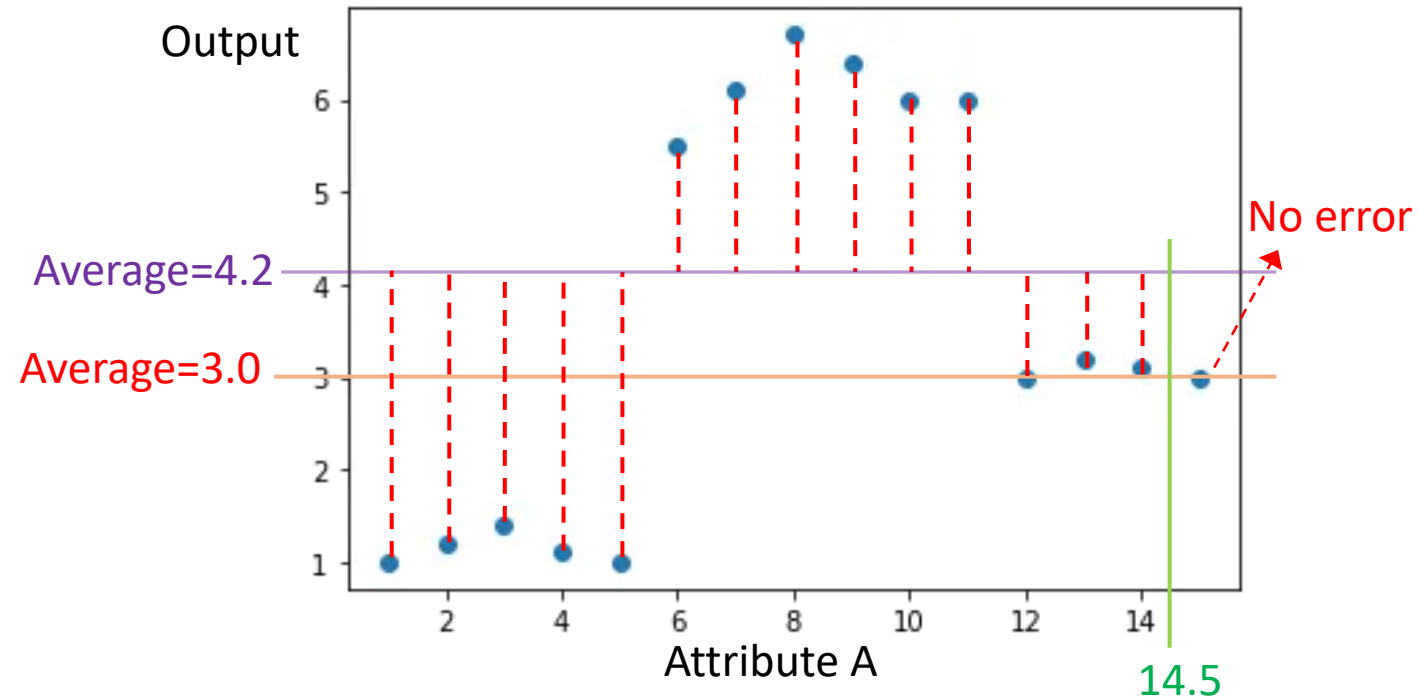
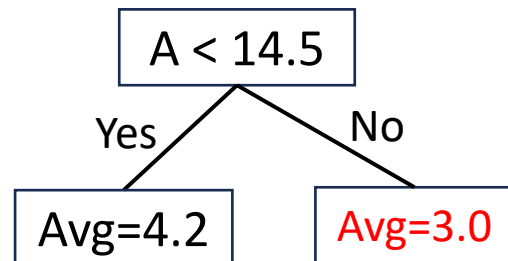
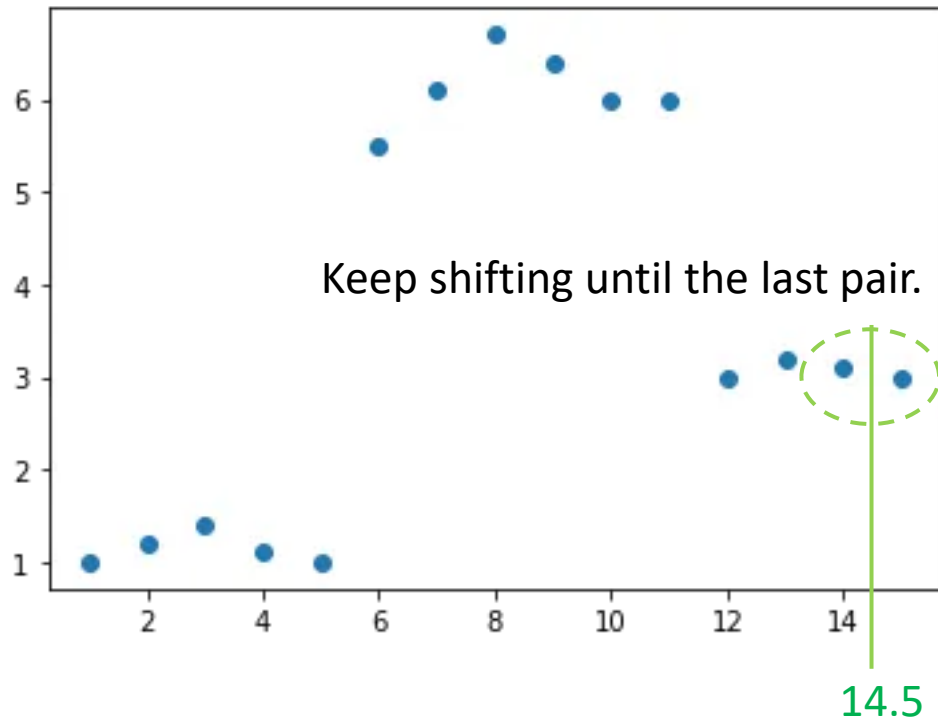
Avg. outputs of 3 points
on the left.

Avg. outputs of 12 points on the right.



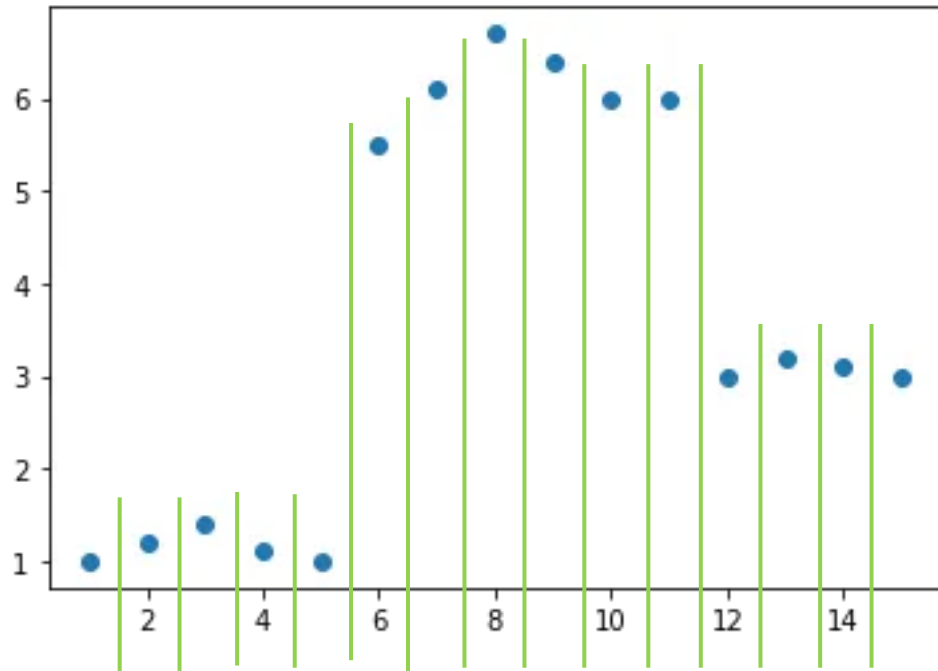
- Calculate the sum-squared error of all data points. Let's call it **E3**.

Applying ID3 to a Regression Problem (Regression Tree)

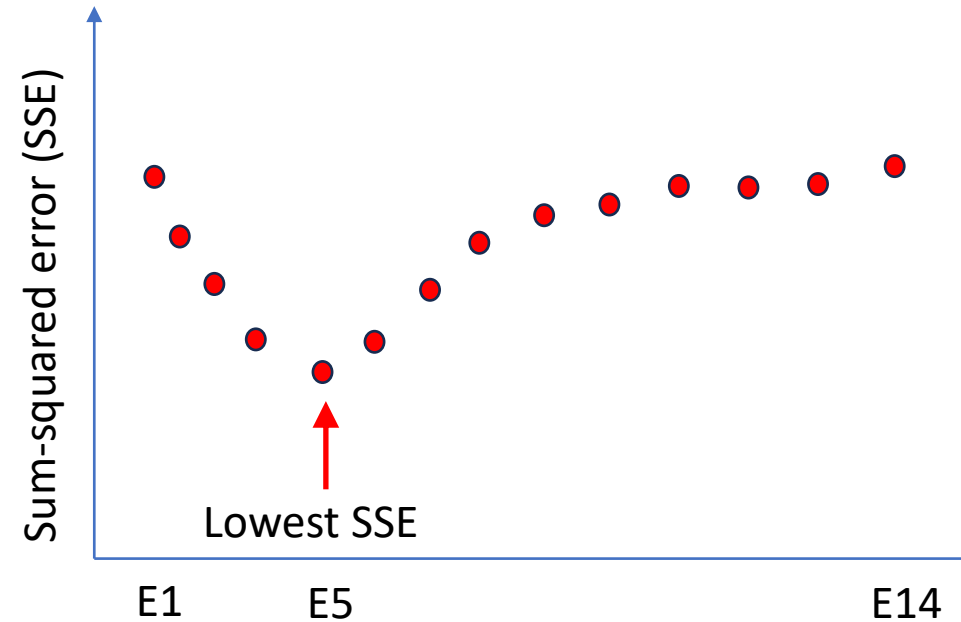
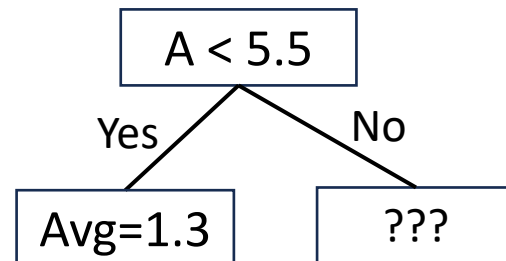


- Calculate the sum-squared error of all data points. Let's call it **E14**.

Applying ID3 to a Regression Problem (Regression Tree)



Best threshold



- Discard all points on the left of the threshold.
- Repeat the process to find the next threshold.

Random Forest Algorithm

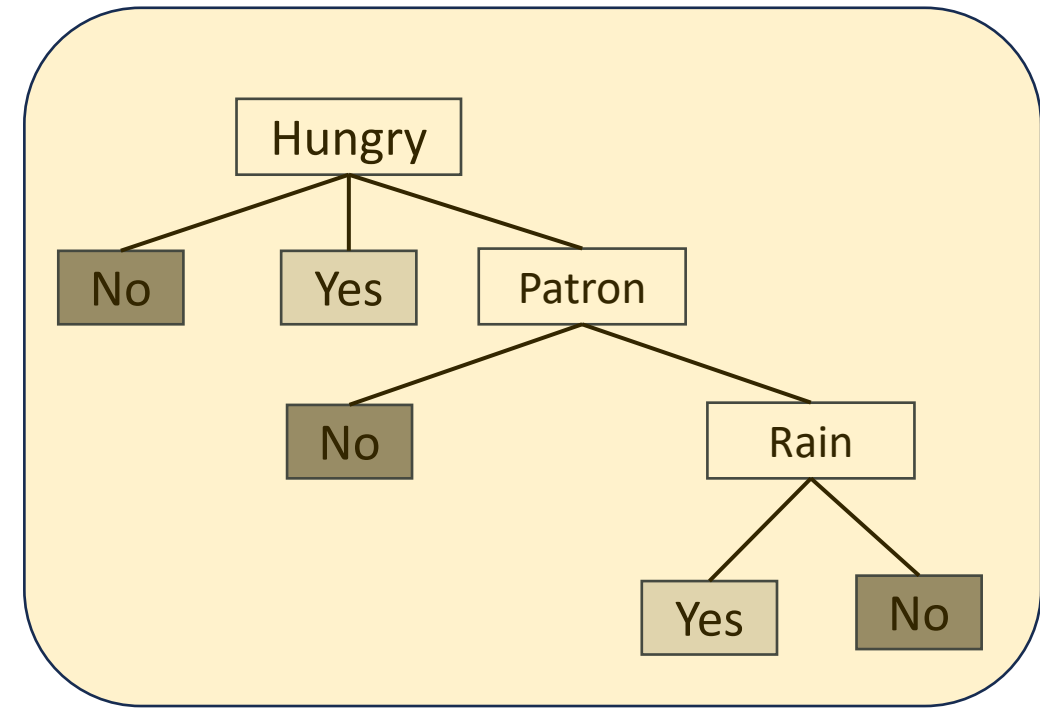
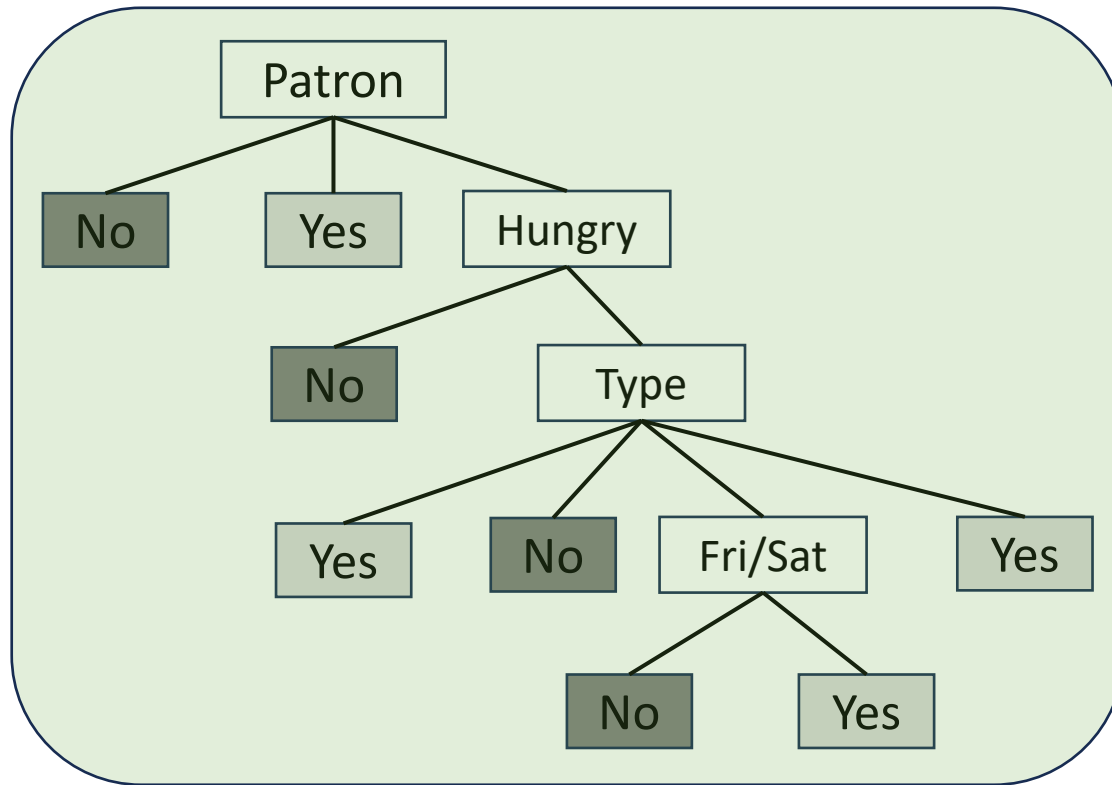
Two major problems of ID3 (Decision tree algorithm):

1. It is easily overfitted to the training data.
2. It is sensitive to the small change in the training data.

Solution:

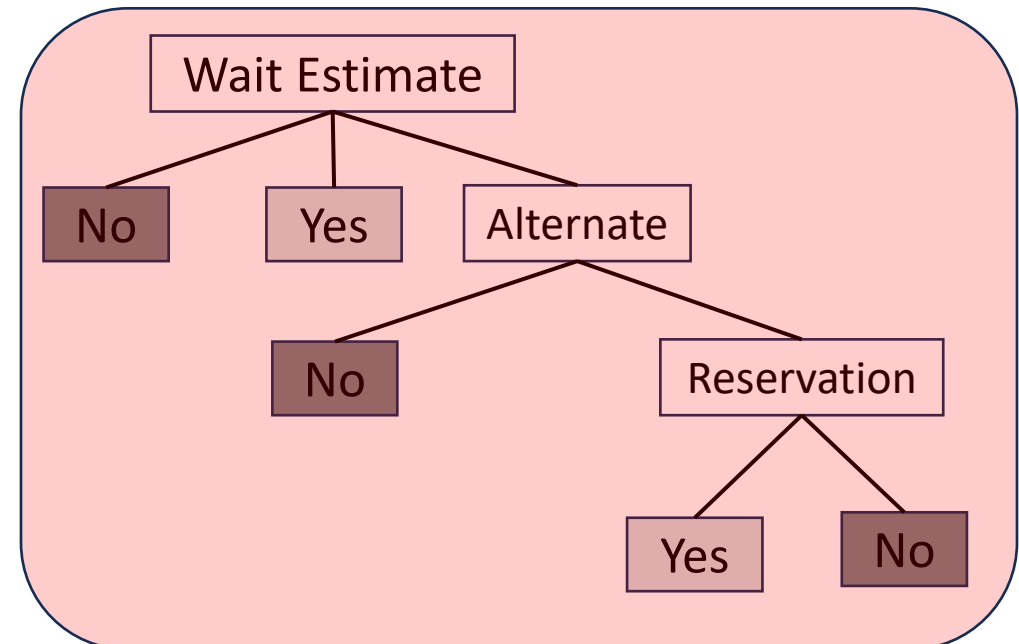
Multiple decision trees.





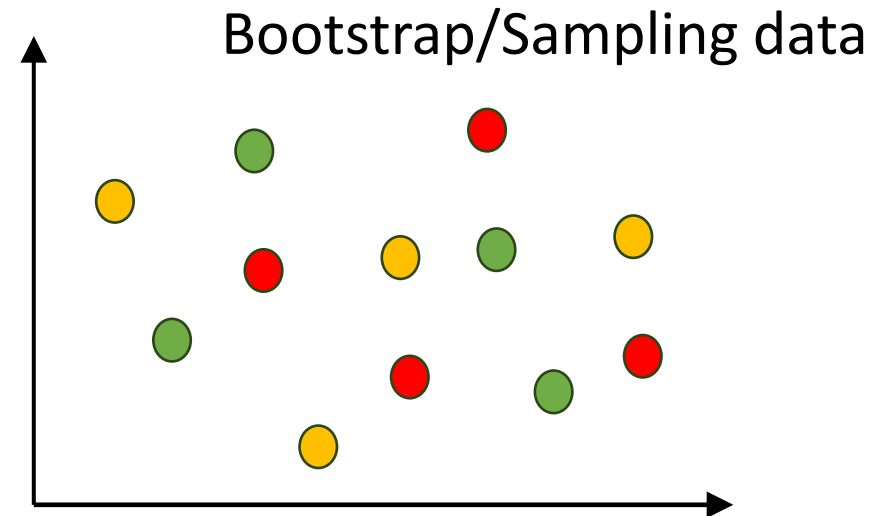
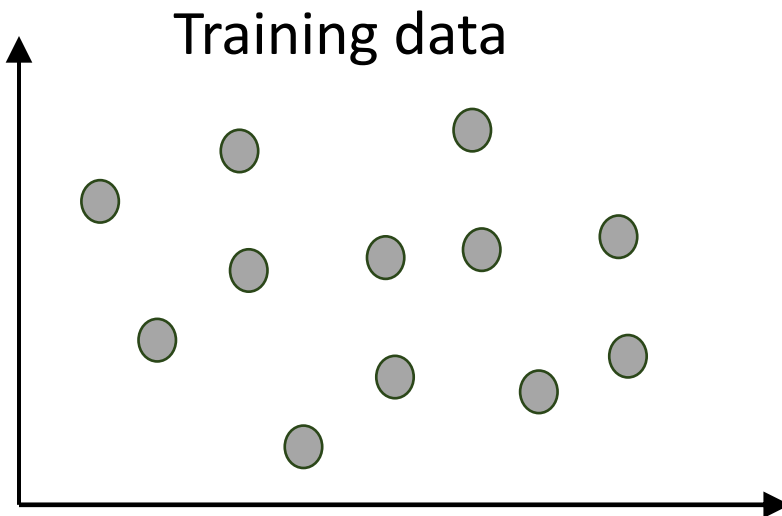
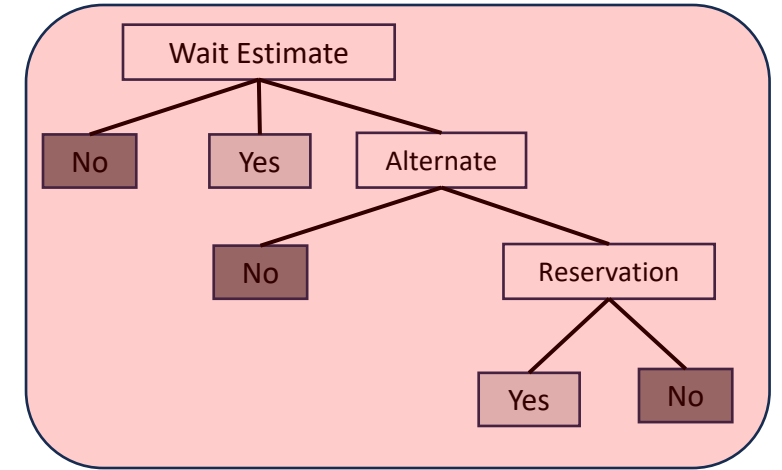
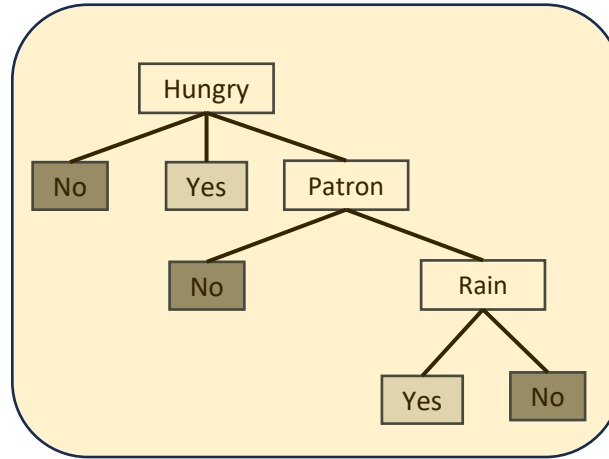
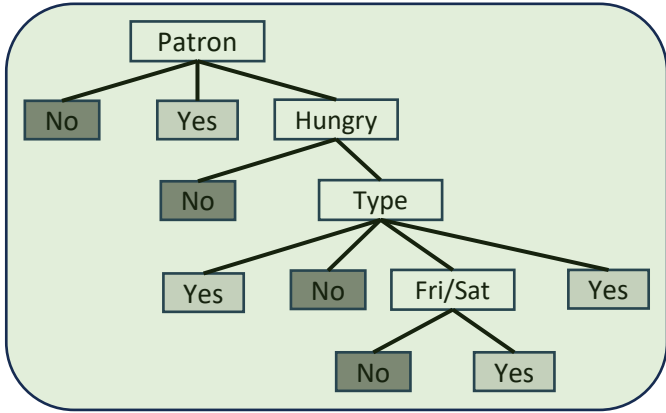
Question:

Why there are many different decision trees for the same training set?

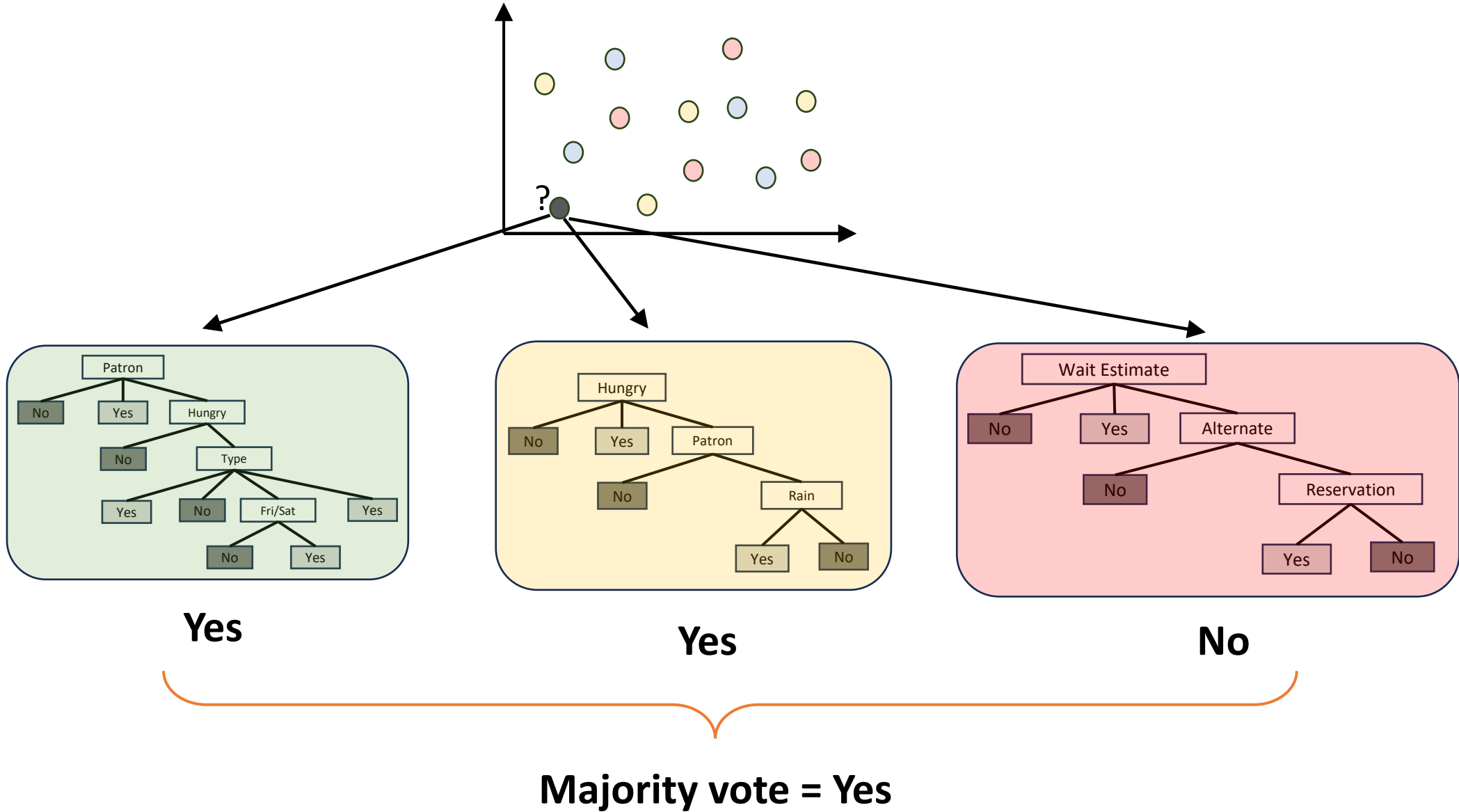


Answer:

Each Decision Tree model in a Random Forest algorithm receives a different subset of training data (bootstrap or sampling).



Prediction on an unseen data point



Advantages of the Random Forest Algorithm

- Robust and less sensitive to small changes to our data. Since it aggregates the predictions of all decision tree models.

Similar to the decision tree algorithm, Random Forest can be applied to classification and regression problems.

K Nearest Neighbors Algorithm (KNN)

- **K nearest neighbors** is a nonparametric algorithm that classifies unseen data based on a distance function.
- An example is classified by a **majority vote of its K neighbors**.

Distance functions
Continuous feature

Euclidean

$$\sqrt{\sum_{i=1}^k (x_i - y_i)^2}$$

← Dimensions

Manhattan

$$\sum_{i=1}^k |x_i - y_i|$$

Minkowski

$$\left(\sum_{i=1}^k (|x_i - y_i|)^q \right)^{1/q}$$

Hamming Distance
Discrete feature

$$D_H = \sum_{i=1}^k |x_i - y_i|$$

$$x = y \Rightarrow D = 0$$

$$x \neq y \Rightarrow D = 1$$

X	Y	Distance
Male	Male	0
Male	Female	1

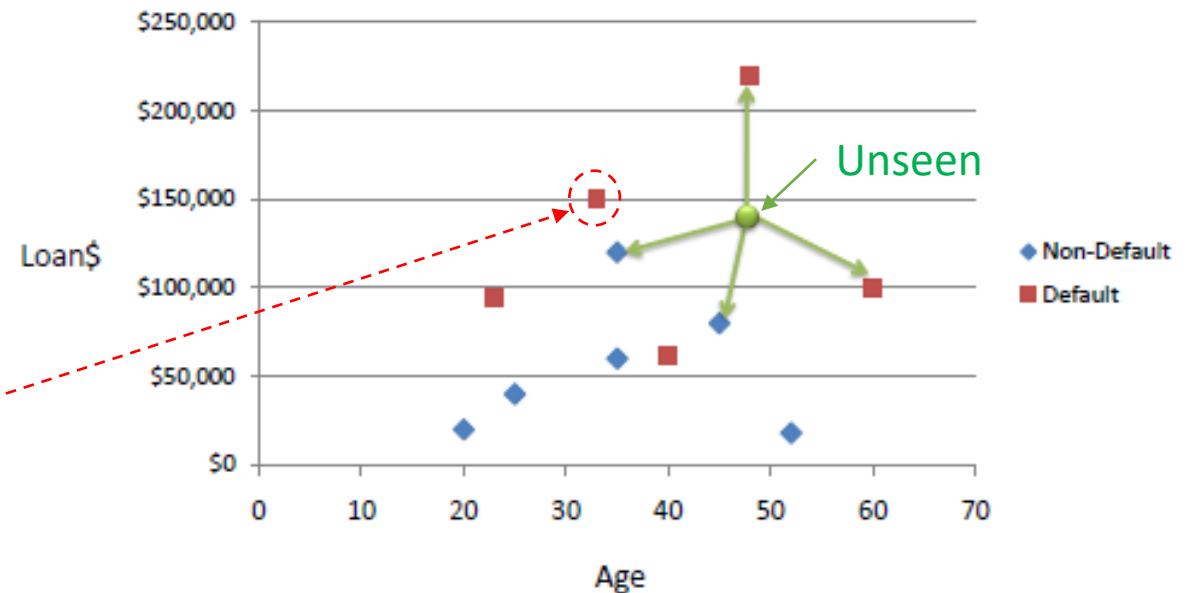
- **Example:** Consider the following data concerning **credit default**. **Age** and **Loan** are two numerical attributes and **Default** is the target class.
- Given **an unseen example (Age=48 and Loan=\$142,000)**

The Shortest Euclidean distance (K=1) = $\sqrt{(48 - 33)^2 + (142000 - 150000)^2} = 8,000 >> \text{Default}=Y$

Age	Loan	Default	Distance
25	\$40,000	N	102000
35	\$60,000	N	82000
45	\$80,000	N	62000
20	\$20,000	N	122000
35	\$120,000	N	22000
52	\$18,000	N	124000
23	\$95,000	Y	47000
40	\$62,000	Y	80000
60	\$100,000	Y	42000
48	\$220,000	Y	78000
33	\$150,000	Y	8000
48	\$142,000	?	

Euclidean Distance

$$D = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$



- With **K=3**, there are two Default=Y and one Default=N closest neighbors. The prediction for the unknown case is again **Default=Y**.

Standardized Distance

- One major drawback in calculating distance measures directly from the training set is in the case where variables have **different measurement scales** or there is a mixture of **numerical and categorical variables**.
- For example, if one variable is based on annual **income in dollars**, and the other is based on **age in years** then **income will have a much higher influence** on the distance calculated.
- One solution is to **standardize the training set**.
- Using the standardized distance on the same training set, the unknown example returned a different neighbor.

Age	Loan	Default	Distance
0.125	0.11	N	0.7652
0.375	0.21	N	0.5200
0.625	0.31	N	0.3160
0	0.01	N	0.9245
0.375	0.50	N	0.3428
0.8	0.00	N	0.6220
0.075	0.38	Y	0.6669
0.5	0.22	Y	0.4437
1	0.41	Y	0.3650
0.7	1.00	Y	0.3861
0.325	0.65	Y	0.3771
0.7	0.61	?	

Standardized Variable

$$X_s = \frac{X - Min}{Max - Min}$$

Evaluating the hypothesis

- We can simply use the error rate to measure the hypothesis.
- Error rate =
$$\frac{\text{Number of examples which are wrong predicted}}{\text{Total number of examples}}$$
- However, a hypothesis h has a low error rate on the training set does not mean that it will generalize well.
- To get a more accurate evaluation, we need to test the hypothesis on an unseen set (test set) of examples.

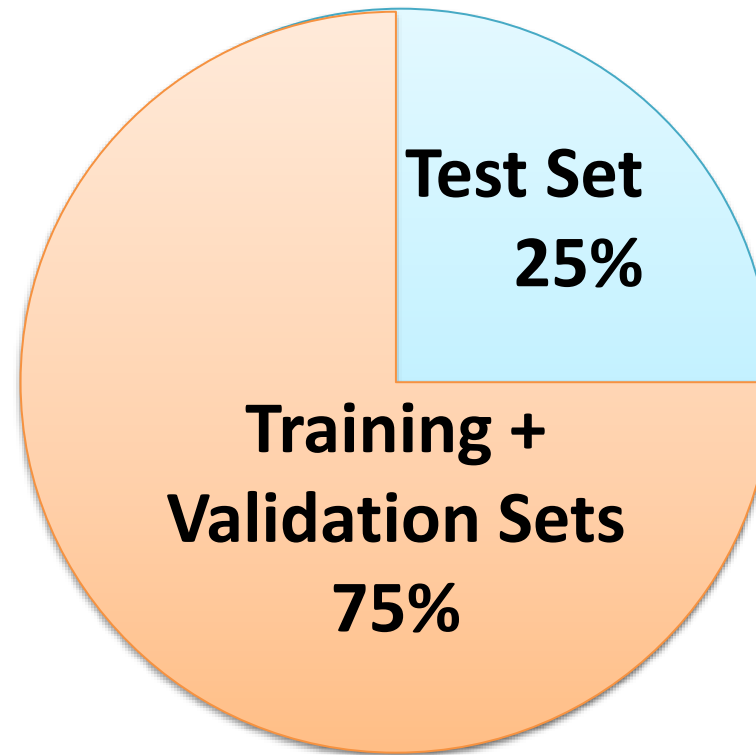
- **Holdout cross-validation** : (If we have a large amount of dataset)

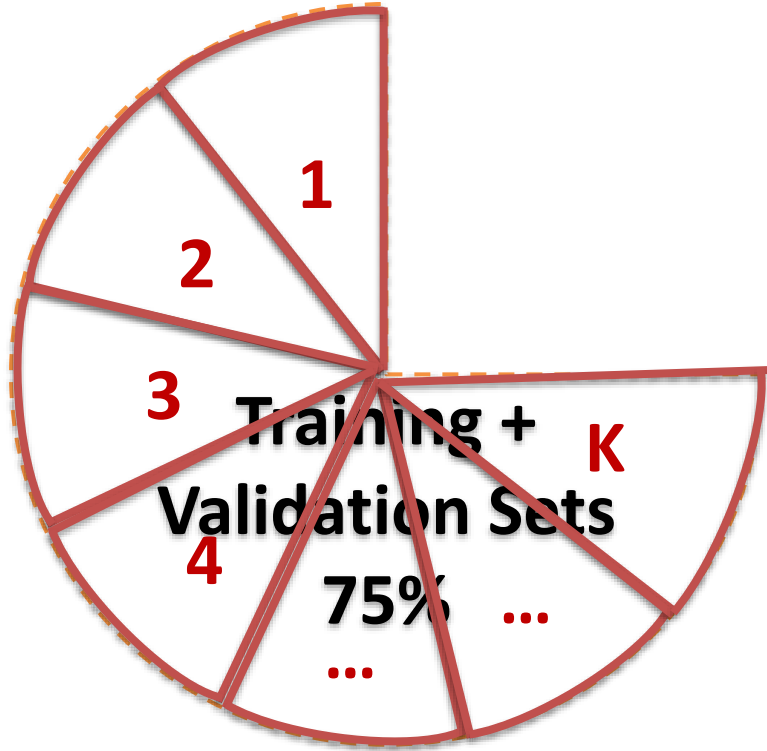
Randomly split the available data into a training set (e.g. 75%) and a test set (e.g. 25%).

- **K-fold cross-validation**: (If we have a limited amount of dataset)

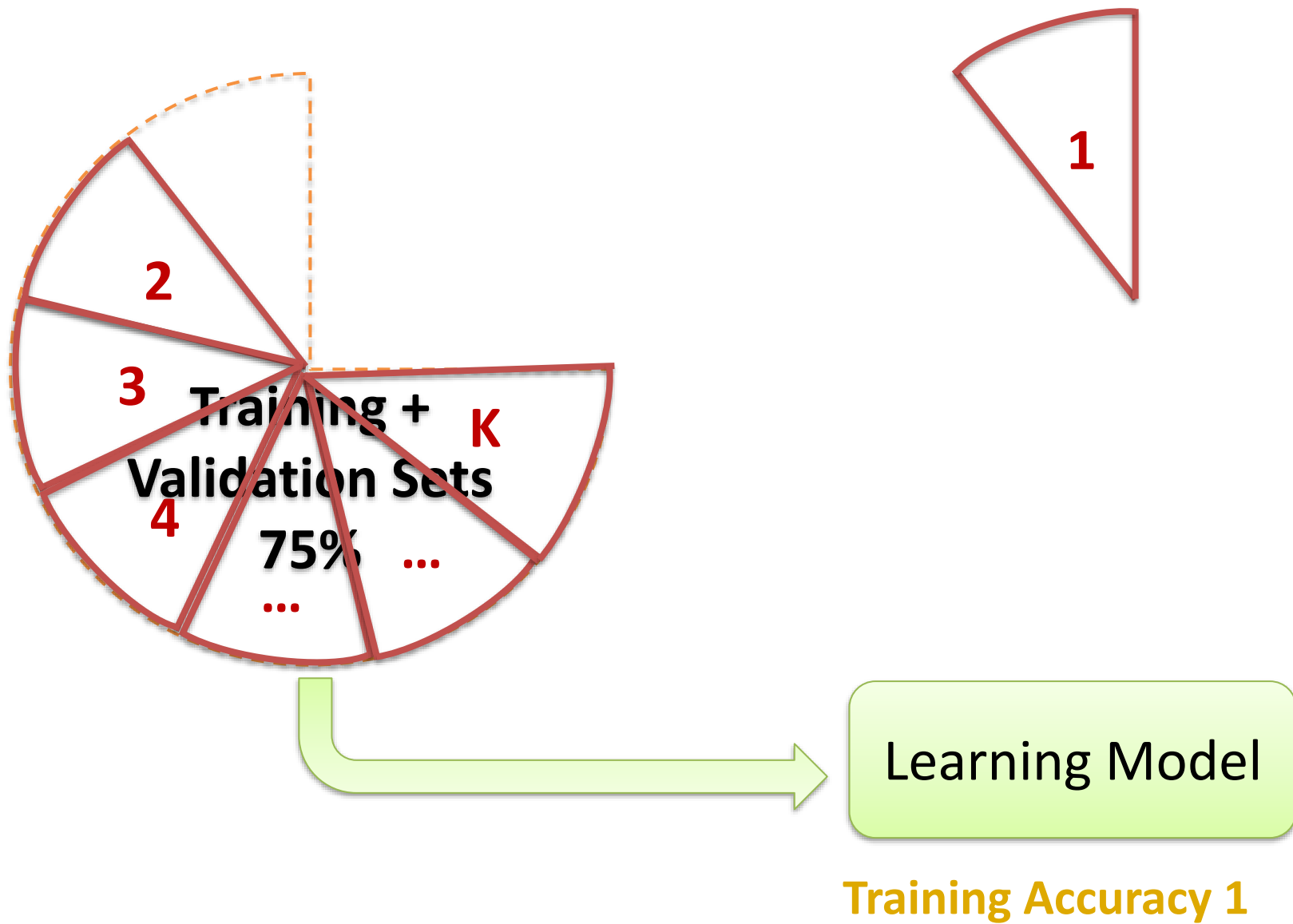
- Split the training data into k equal subsets.
- Then perform k rounds of learning. On each round, $1/k$ of the data is held out as a validation set and the remaining examples are used as training data.
- The average error rate of the k rounds is finally calculated. Popular values for k are 5 and 10. The extreme is $k=n$ (the total number of examples), also known as **leave-one-out cross-validation** or **LOOCV**.

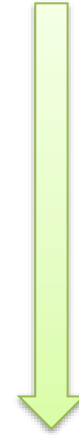
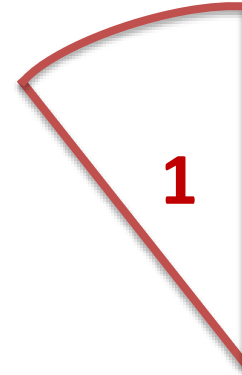
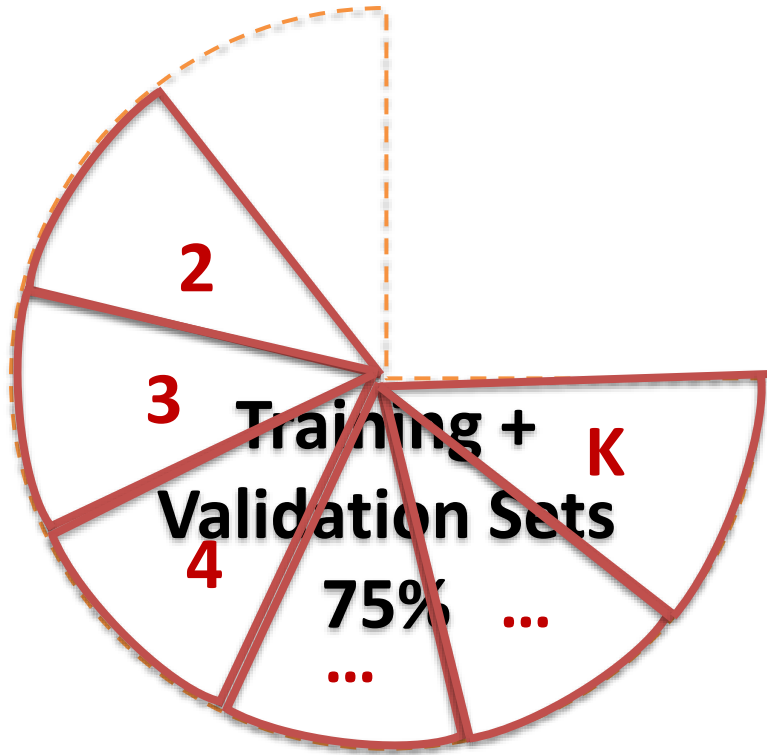
K-Fold Cross Validation





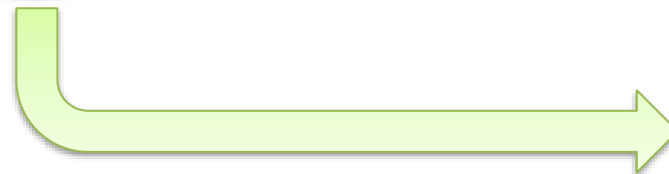
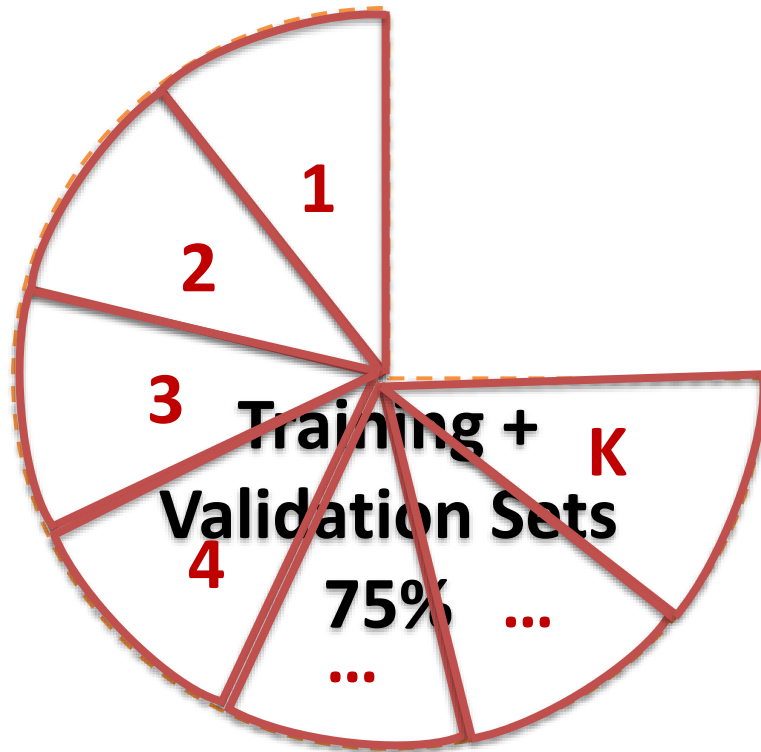
Learning Model





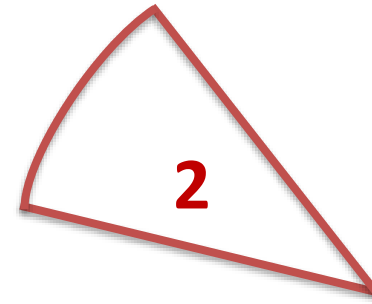
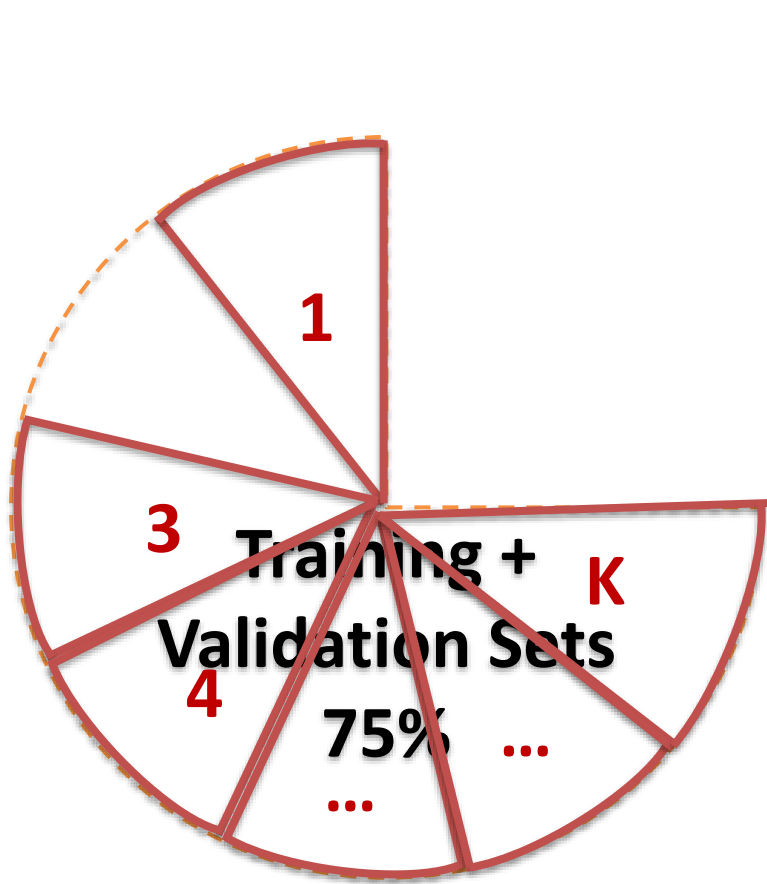
Learning Model

Validation Accuracy 1



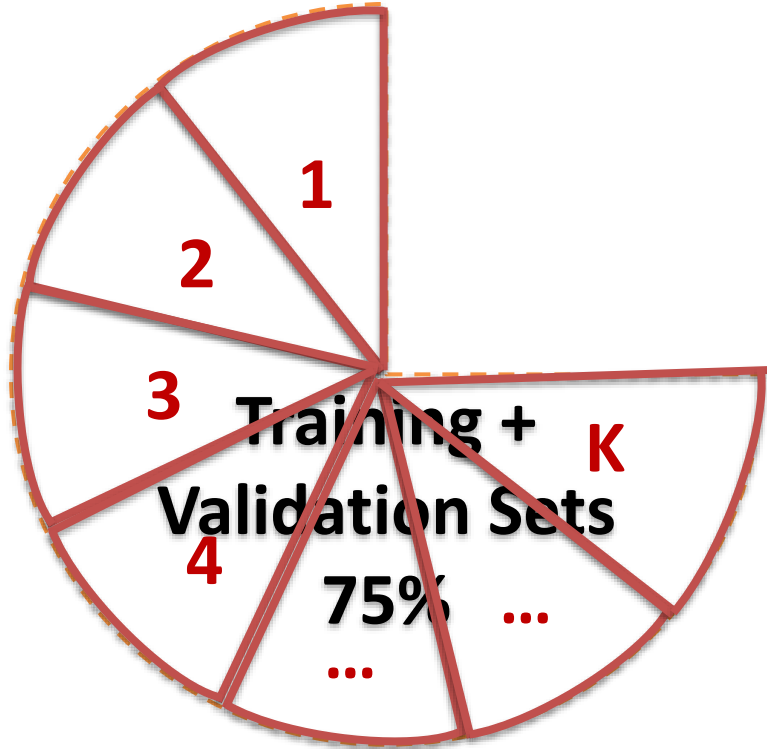
Learning Model

Training Accuracy 2

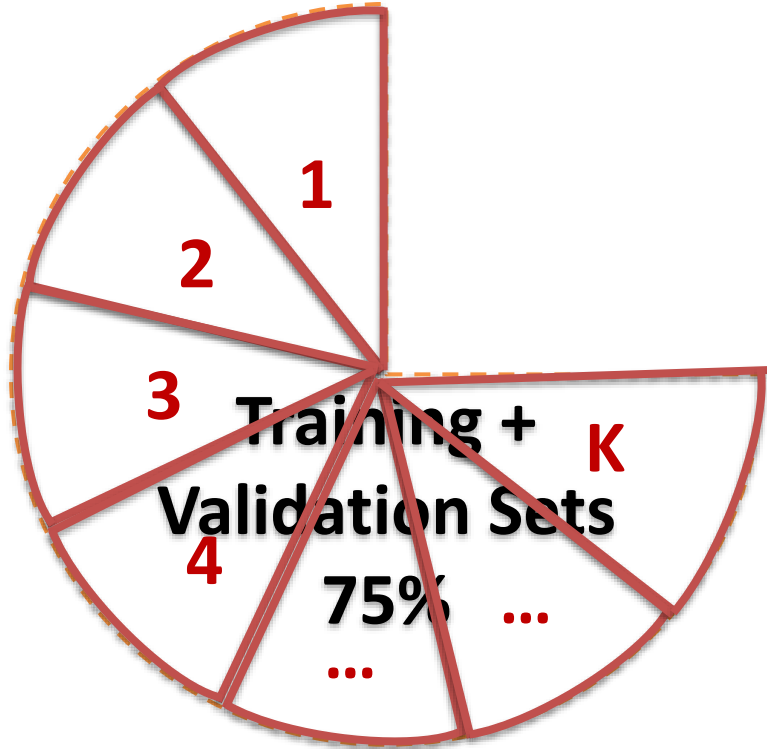


Learning Model

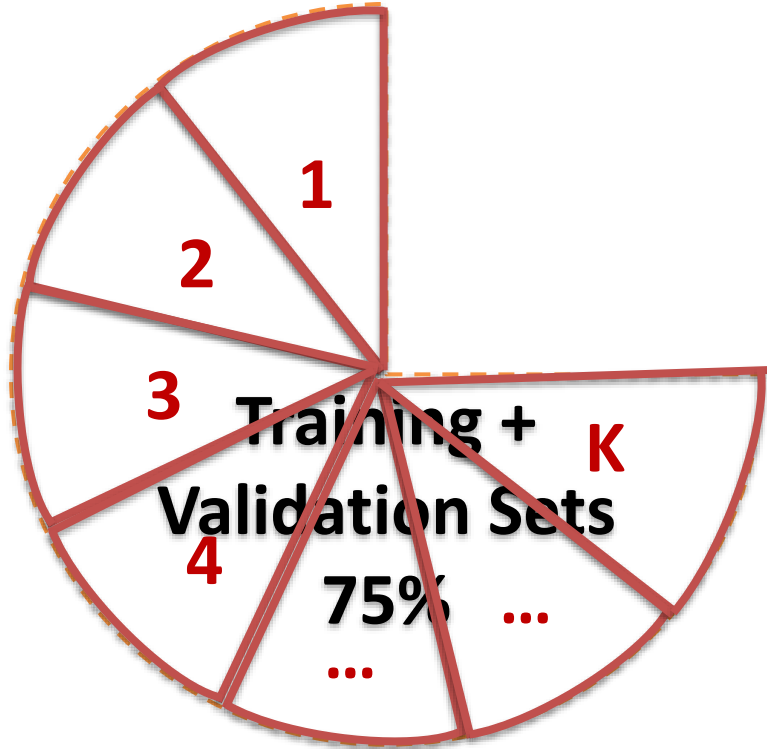
Validation Accuracy 2



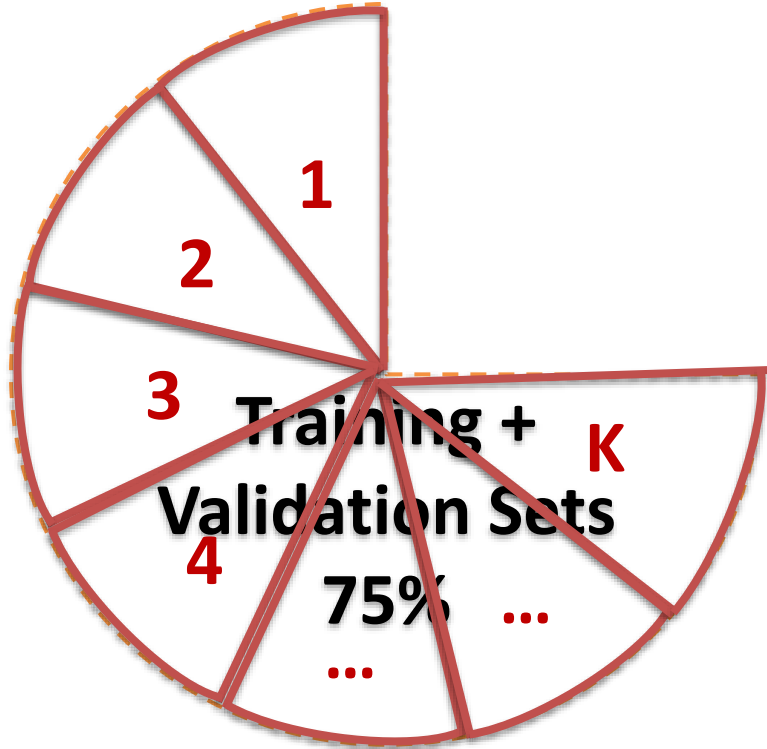
Learning Model



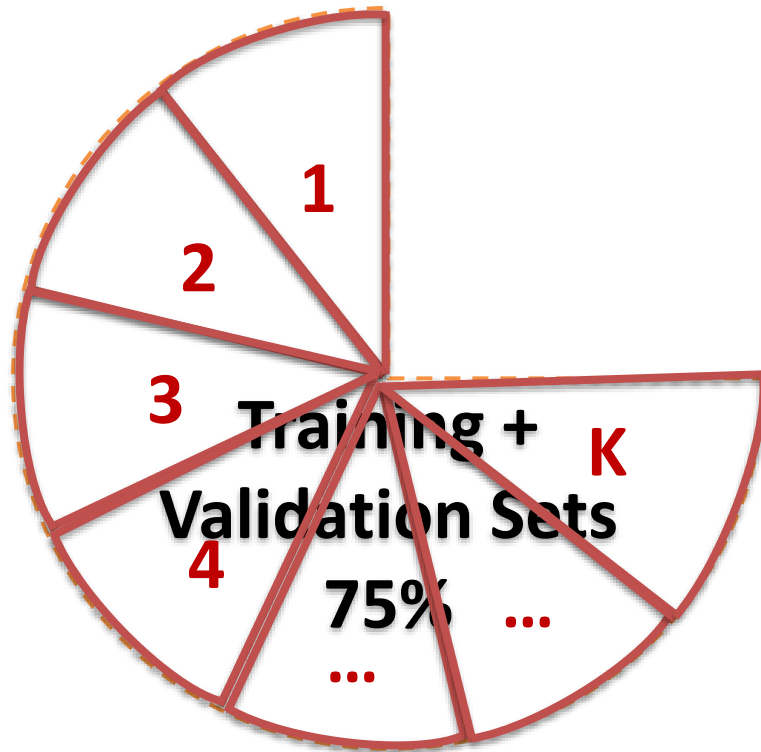
Learning Model



Learning Model

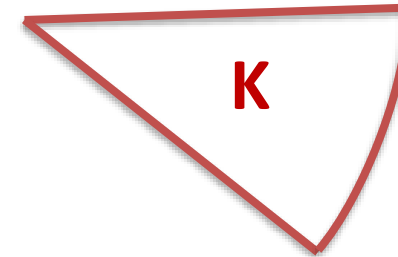
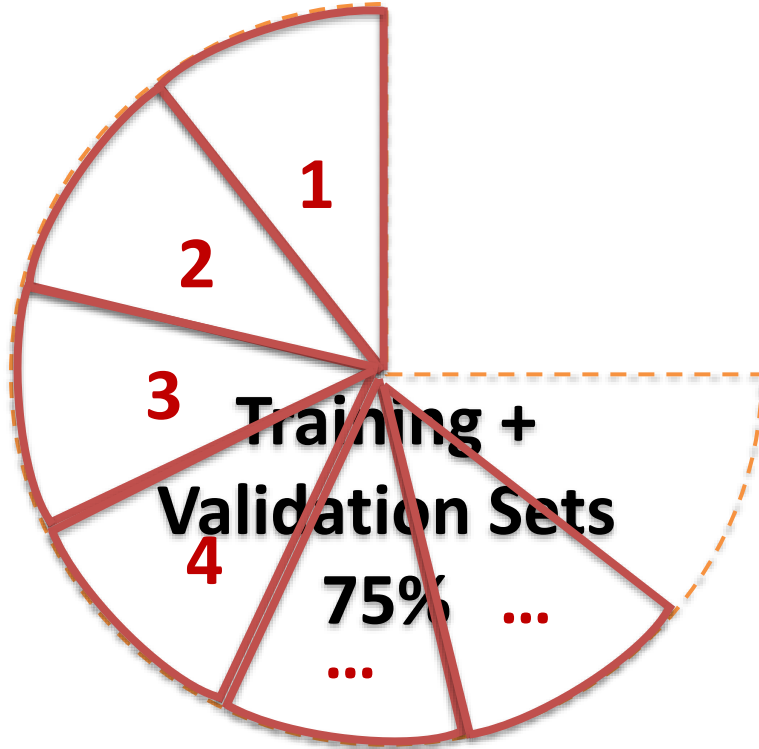


Learning Model



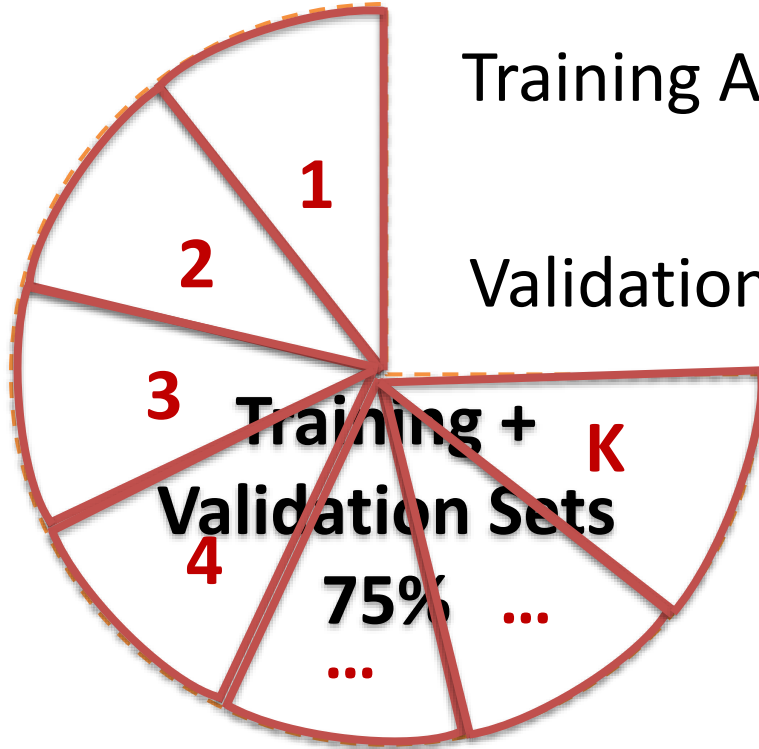
Learning Model

Training Accuracy K



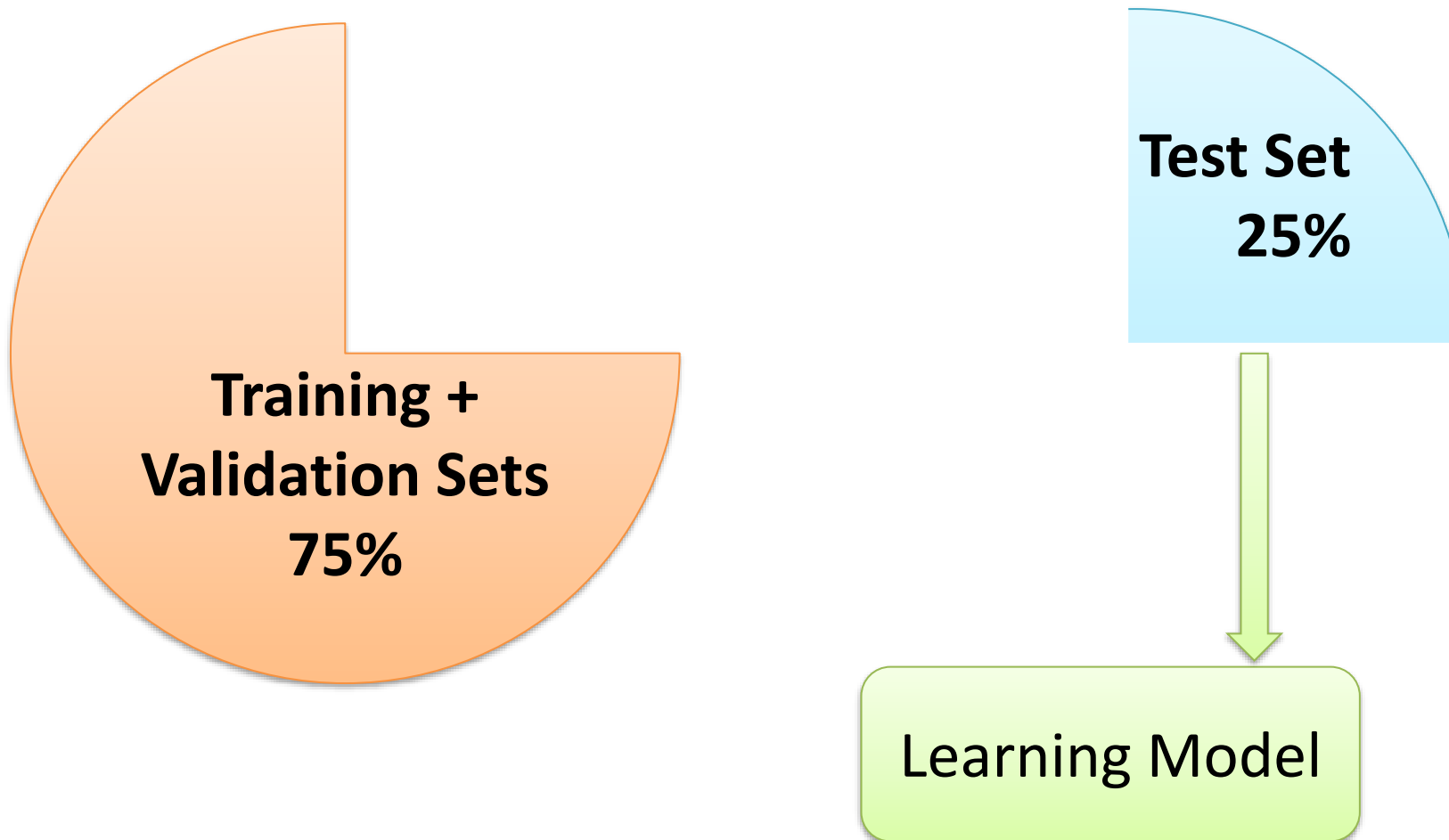
Learning Model

Validation Accuracy K



$$\text{Training Accuracy} = (\sum_{i=1}^K \text{Train_Acc_}i)/K$$

$$\text{Validation Accuracy} = (\sum_{i=1}^K \text{Valid_Acc_}i)/K$$



Note: Percentage of test set can be varied, depends on how many total of instances we have.

Test Accuracy